AI 時代的細部設計

原理、設計典範與重構

軟體工程（IM 6003）｜接在 Ch9、Ch10、Ch11、Ch16、Ch17 之後

取代去年的兩份補充：Software Detailed Design — Order Placement；Refactoring & Design Patterns

主線例子：下單系統（Order Placement）＋ AI 點餐助理；對照：教科書的圖書館借書

這一講的地圖

1	**原理與五種設計層次**	解耦與內聚的操作型定義；依賴反轉；同一個下單 use case 的五種設計
2	**細部設計變成給 AI 的規格**	情境表 → 循序圖 → 類別圖 → 交給 AI → 反畫對照
3	**用 AI 重構下單系統（UC1–UC5）**	壞味道 → 安全網 → 一次一個手法 → 循序圖前後對照
4	**熟悉的 pattern，在 AI 功能裡的新角色**	加入「AI 點餐助理」：Bridge、Strategy、Proxy、Command、Factory、Template Method、Composite
5	**持久化的延伸**	關聯式資料 + 向量索引 + 對話記憶；MCP；冪等與 outbox
6	**哪些不變、哪些要重學**	現代版 pattern 地圖；用 pattern 名稱跟 AI 溝通；課堂練習

第 1 部分取代去年的「五種設計選擇」；第 3 部分取代去年的「Refactoring & Design Patterns」；第 4 部分起是 AI 功能本身的設計

學習目標

1. 用「變更影響」與「可測試性」比較設計的好壞，並說明依賴反轉為什麼重要
2. 說出為什麼 AI 產生的程式「能跑卻難維護」，並把 use case 寫成可以交給 AI 的設計規格
3. 在測試保護下，一次用一個手法重構，並用循序圖說明前後差別
4. 在 AI 功能中正確擺放 LLM，並用熟悉的 pattern 處理換模型、快取、工具呼叫與核准
5. 說明向量索引、對話記憶、MCP、冪等在持久化設計上的位置
6. 判斷什麼時候「不要」用 pattern、該停在哪一個設計層次

開場：寫程式變便宜了，設計反而更重要

時效：穩定

觀察

- AI 幾秒就能產生一個能跑的下單功能
- 同樣的需求，問三次會得到三種不同的結構
- 加第二個需求時，AI 傾向「在原地再加一個 if」

推論

- 產生程式的成本趨近零，但「讀、改、測」的成本沒有下降
- 程式量變大，耦合的代價被放大
- 誰決定結構？沒人決定，就是 AI 的預設

結論

- 細部設計從「畫給人看的藍圖」變成「交給 AI 的規格」與「驗收 AI 的標準」
- 解耦、內聚、責任指派這些老原理，比以前更重要

什麼情況下這不成立

- 一次性的腳本、展示用原型：設計投資回收不了
- 模型能長期記住整個系統的設計意圖時，規格的形式會改變（但仍需要有人決定意圖）

兩個例子貫穿全講

主線：下單系統（延續去年的 Order Placement）

- **UC1 下新訂單：**查商品、算金額、存訂單
- **UC2 需要核准的訂單、取消：**大額 B2B 訂單要主管核准；取消要回補庫存
- **UC3 折扣：**打九折、折 50 元……
- **UC4 套餐：**套餐裡有單品，也可以有小套餐
- **UC5 多種付款：**信用卡、LINE Pay，各家 API 長得不一樣
- **第 4 部分加入：AI 點餐助理：**顧客打一句話，助理整理成購物車、問缺的資訊、確認後下單

對照：教科書的圖書館借書（Ch9–Ch11、Ch17）

- **CheckoutGUI → CheckoutController → DBMgr：**Controller、Expert、Creator 的原型
- **Ch17 的 DBMgr：**Bridge、Command、Template Method、Proxy
- **Ch16 的課堂練習：**「agent 會動態產生並執行計畫，計畫由條件動作與子計畫組成」——第 4 部分會回答它
- **本講只用課上教過的 pattern：**Controller、Expert、Creator、Strategy、Factory、Adapter、Bridge、Proxy、Command、Template Method、Composite、Iterator

第 1 部分

原理與五種設計層次

同一個下單 use case，五種細部設計：用「需求改變時要改幾個地方」來比較

- 兩個核心原理：解耦與內聚（操作型定義）；依賴方向與依賴反轉
- 一個 use case、五個固定的變更情境
- 層次 1 單一 Controller → 2 BCE → 3 分層 + Repository → 4 Strategy / Factory / Adapter → 5 Hexagonal
- 比較、什麼時候停在哪一層、每一層能寫什麼測試
- AI 時代的意義：層次名稱是 prompt 的詞彙；依賴規則寫成測試

說明

去年的版本少了什麼？今年怎麼改

時效：穩定

去年的「五種設計選擇」	今年的版本
業務情境列了六種訂單、補償、稽核，五種設計只畫到一部分	範圍收斂成一個 use case + 五個固定的變更情境，每一層都用同一組情境檢驗
比較表用星等（主觀）	改成「變更影響」：每個情境要改哪裡（✓ 只新增 / △ 小改 / ✗ 牽一髮動全身）
循序圖是 PlantUML 截圖，字很小	PowerPoint 原生循序圖，紅色標出每一層的問題點
沒有說明依賴的方向	加入依賴反轉：第 4 層與第 5 層真正的差別
暗示「越後面越好」	加入「什麼時候停在哪一層」：設計要和規模、變化頻率相稱
測試的部分是對話截圖	整理成「每一層能寫哪些測試」，並附單元測試範例
GPT prompt 只要求「產生五種設計」	改成「產生後，用變更情境自己審查」，並加上架構測試

兩個核心原理：解耦與內聚（操作型定義）

時效：穩定

解耦（Low Coupling）

- **定義**：改 A 的時候，B 不需要跟著改
- **操作型問法**：一個需求改變，要動幾個檔案？
- **另一個測法**：能不能不連資料庫、不呼叫外部 API 就測試核心邏輯？
- **常見誤會**：分層不等於解耦：分了層，但上層直接 new 下層的具體類別，仍然綁死

內聚（High Cohesion）

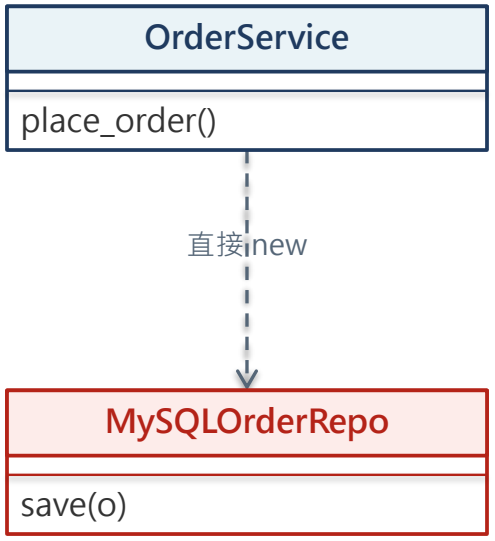
- **定義**：一個模組裡的東西，都為同一件事而存在
- **操作型問法**：這個類別有幾個「改變的理由」？
- **例子**：OrderController 若同時因「折扣規則」「付款方式」「資料庫欄位」而改，就是低內聚
- **常見誤會**：放在同一個檔案不等於內聚：要讓模組裡只做一件清楚的事

本部分的所有比較，都只用這兩把尺：變更影響（解耦）與改變的理由（內聚）

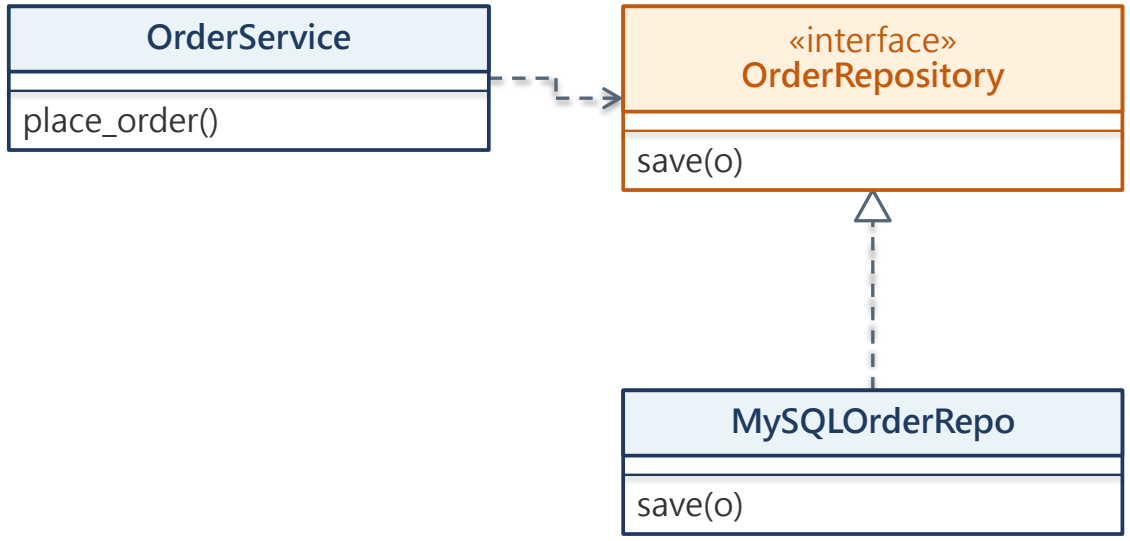
依賴方向與依賴反轉（Dependency Inversion）

時效：穩定

✗ 高層依賴低層的細節



✓ 兩者都依賴抽象；介面由「使用者」這一方定義



為什麼重要

換資料庫、換付款、換 LLM 供應商、換成測試用的假物件——全靠這一個箭頭方向。第 5 層（Hexagonal）就是把它做到底。

情境

一個 use case、五個固定的變更情境

時效：穩定

UC：下單（本部分的範圍）

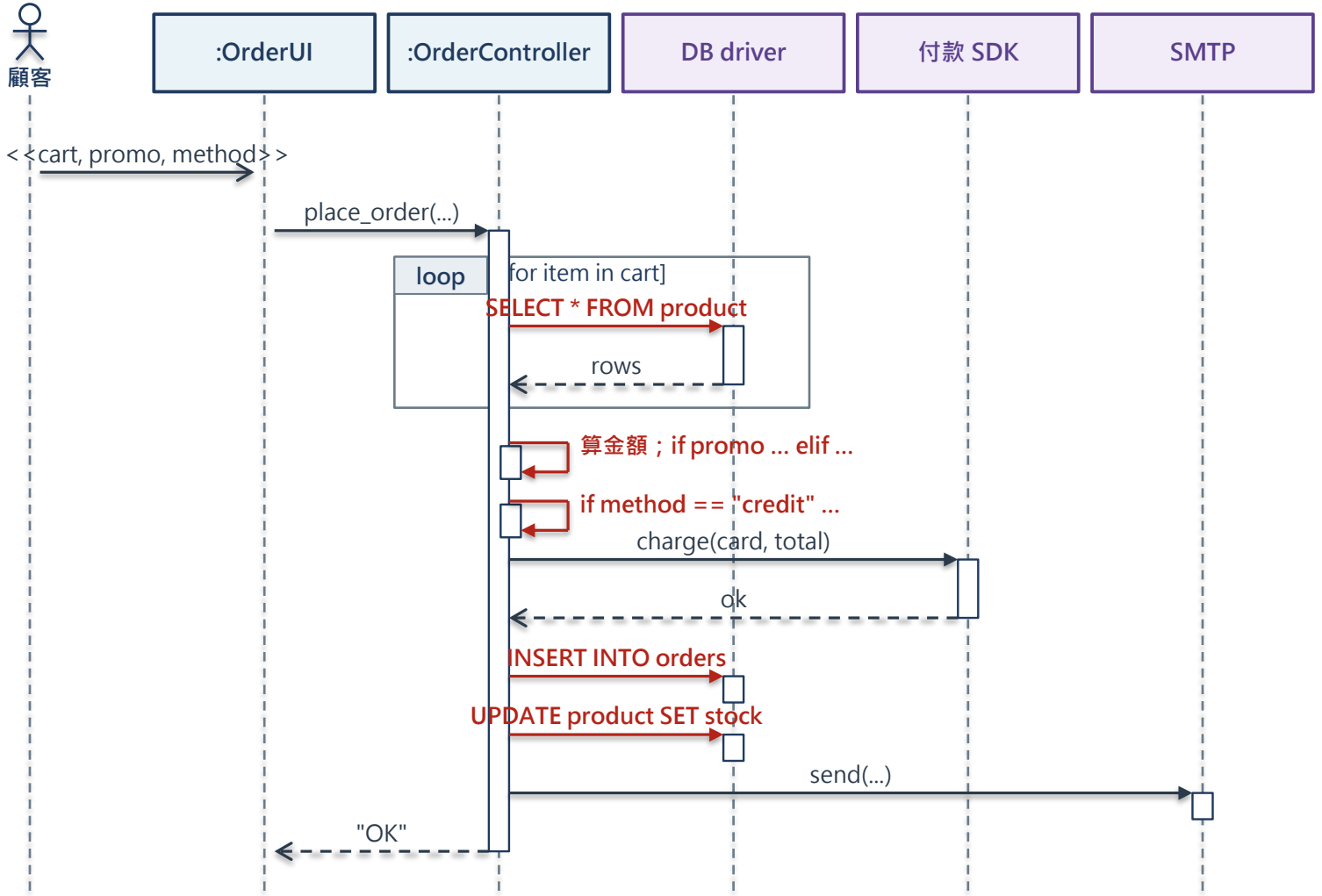
- 顧客送出購物車與付款方式
- 系統查商品與庫存
- 依促銷代碼計算折扣後金額
- 向付款服務請款
- 付款成功才儲存訂單、扣庫存
- 寄出訂單通知
- 資料都在資料庫，透過 database driver 存取

#	變更情境	它在測什麼
C1	加一種折扣（滿千送百）	業務規則的擴充點
C2	加一種付款（街口支付）	外部服務的隔離
C3	換資料庫（MySQL → PostgreSQL）	資料存取的隔離
C4	加一個入口（App API、AI 助理）	流程能不能重用
C5	不連資料庫與外部 API 就能測試	可測試性（解耦的另一面）

五層設計都實作同一個 use case，用同樣五個情境檢驗：✓ 只要新增　△ 要小改　✗ 牽一髮動全身

層次 1：單一 Controller 完成所有工作

時效：穩定



- **特徵**：沒有分層，Controller 直接讀寫資料庫、呼叫 SDK
- **一個方法的改變理由**：折扣、付款、欄位、庫存規則、通知格式.....
- **它從哪裡來**：這正是 AI 只聽一句話時的預設寫法（第 2 部分）
- **不是不能用**：一次性的展示原型可以接受

層次 1 的變更檢驗

時效：穩定

變更情境	要改的地方	
C1 加折扣	改 place_order 的 if 鏈，重測全部	✗
C2 加付款	改 place_order 的 if 鏈	✗
C3 換資料庫	改散在流程中的每一段 SQL	✗
C4 加入口	把整段流程複製到新的入口	✗
C5 測試	一定要真的資料庫與付款服務	✗

代價：只有 1 個類別；但每個需求都要改它

改善了什麼

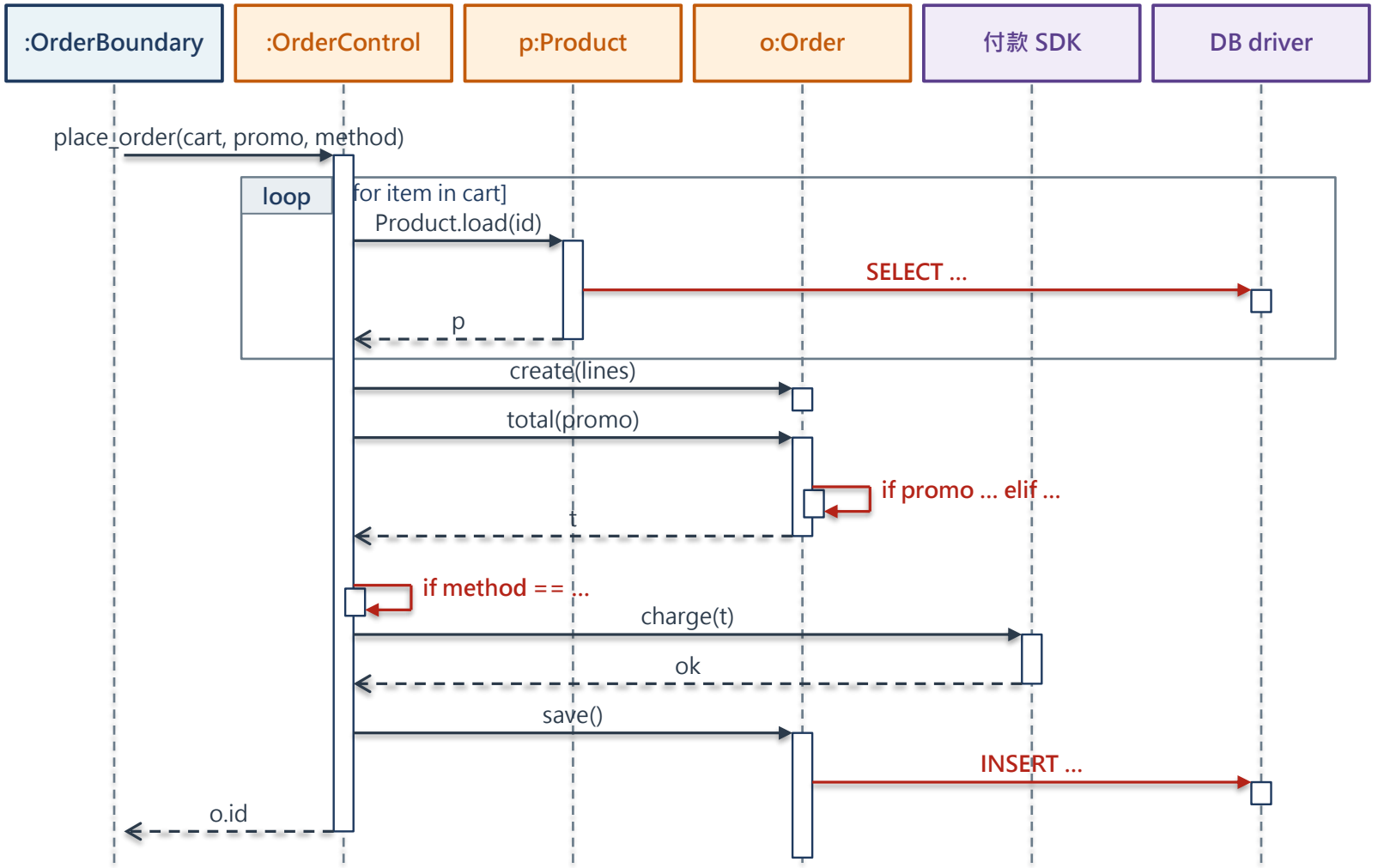
- 寫得最快
- 流程一眼看完（在小規模時）

還剩下的問題

- 介面、流程、規則、資料存取全部混在一起
- 沒有任何擴充點
- 無法單元測試

層次 2：BCE（Boundary、Control、Entity）

時效：穩定



- **Boundary**：只處理介面
- **Control**：一個 use case 一個，負責流程（Ch10 的 Controller pattern）
- **Entity**：Order 自己算金額（Expert）
- **紅色**：Entity 自己讀寫資料庫；折扣與付款仍是 if 鏈

層次 2 的變更檢驗

時效：穩定

變更情境	要改的地方	
C1 加折扣	改 Order.total 的 if 鏈	✗
C2 加付款	改 OrderControl 的 if 鏈	✗
C3 換資料庫	改每一個 Entity 裡的 SQL	✗
C4 加入口	新 Boundary 直接呼叫同一個 Control	✓
C5 測試	Entity 綁著資料庫，難以單獨測試	△

代價：約 5 個類別；Entity 變胖

改善了什麼

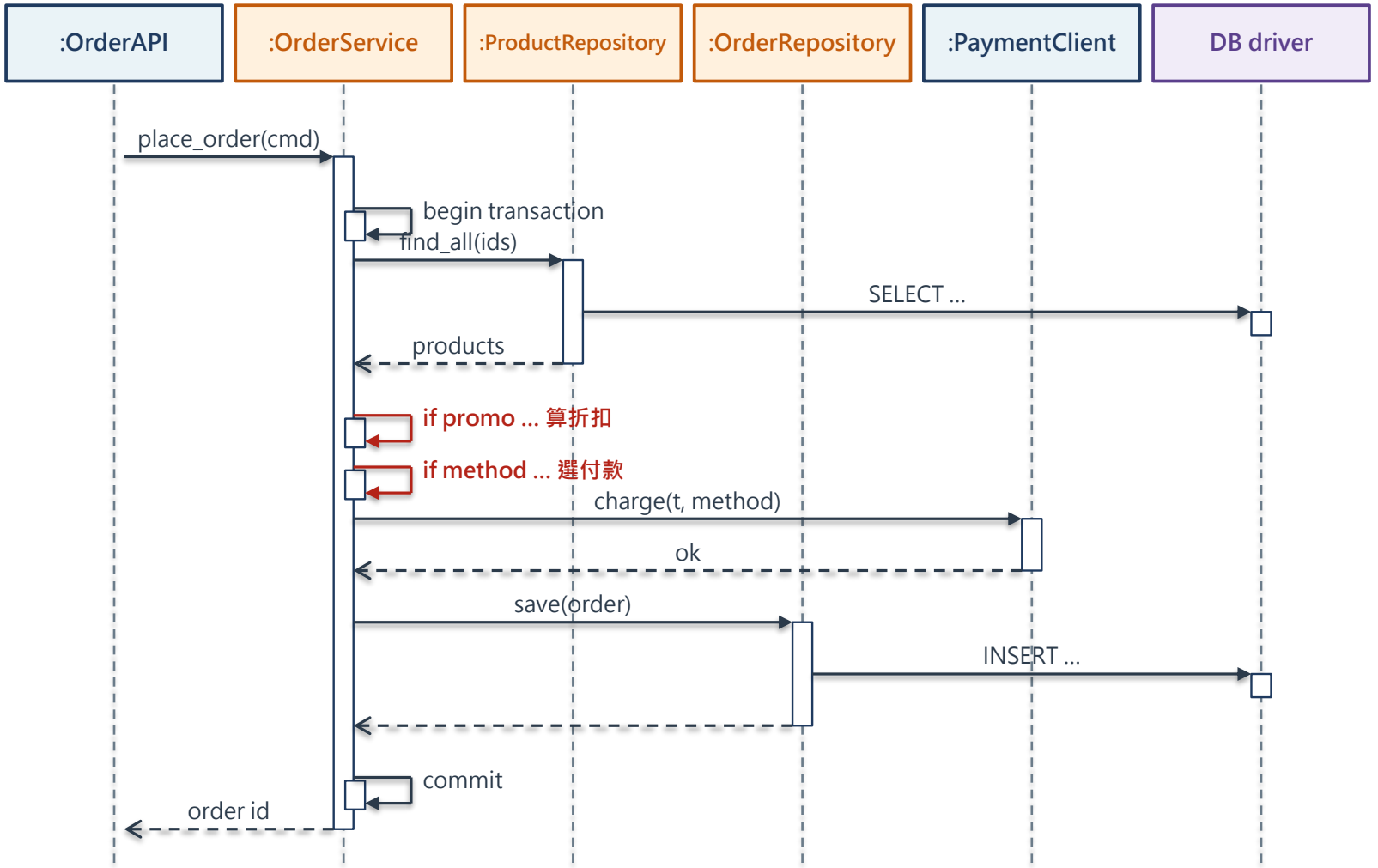
- 介面與流程分開：多一個入口不用複製流程
- 金額由 Order 計算，內聚提高

還剩下的問題

- Entity 同時是「業務規則」與「資料庫存取」（兩個改變理由）
- 型別分支仍在

層次 3：分層 + Repository

時效：穩定



- **API → Service → Repository → DB**：每層只呼叫下一層
- **Repository**：SQL 全部集中；Service 只看到物件（Ch17 的 DBMgr 精神）
- **Service 管交易**：付款成功才 commit
- **紅色**：折扣與付款的分支跑到 Service 裡，Service 開始臃腫

層次 3 的變更檢驗

時效：穩定

變更情境	要改的地方	
C1 加折扣	改 OrderService 的 if 鏈	✗
C2 加付款	改 OrderService 與 PaymentClient	✗
C3 換資料庫	只改 Repository 的實作	✓
C4 加入口	新入口呼叫同一個 Service	✓
C5 測試	Service 若自己 new Repository，就換不成假的	△

代價：約 7 個類別；多數 CRUD 系統停在這裡就夠了

改善了什麼

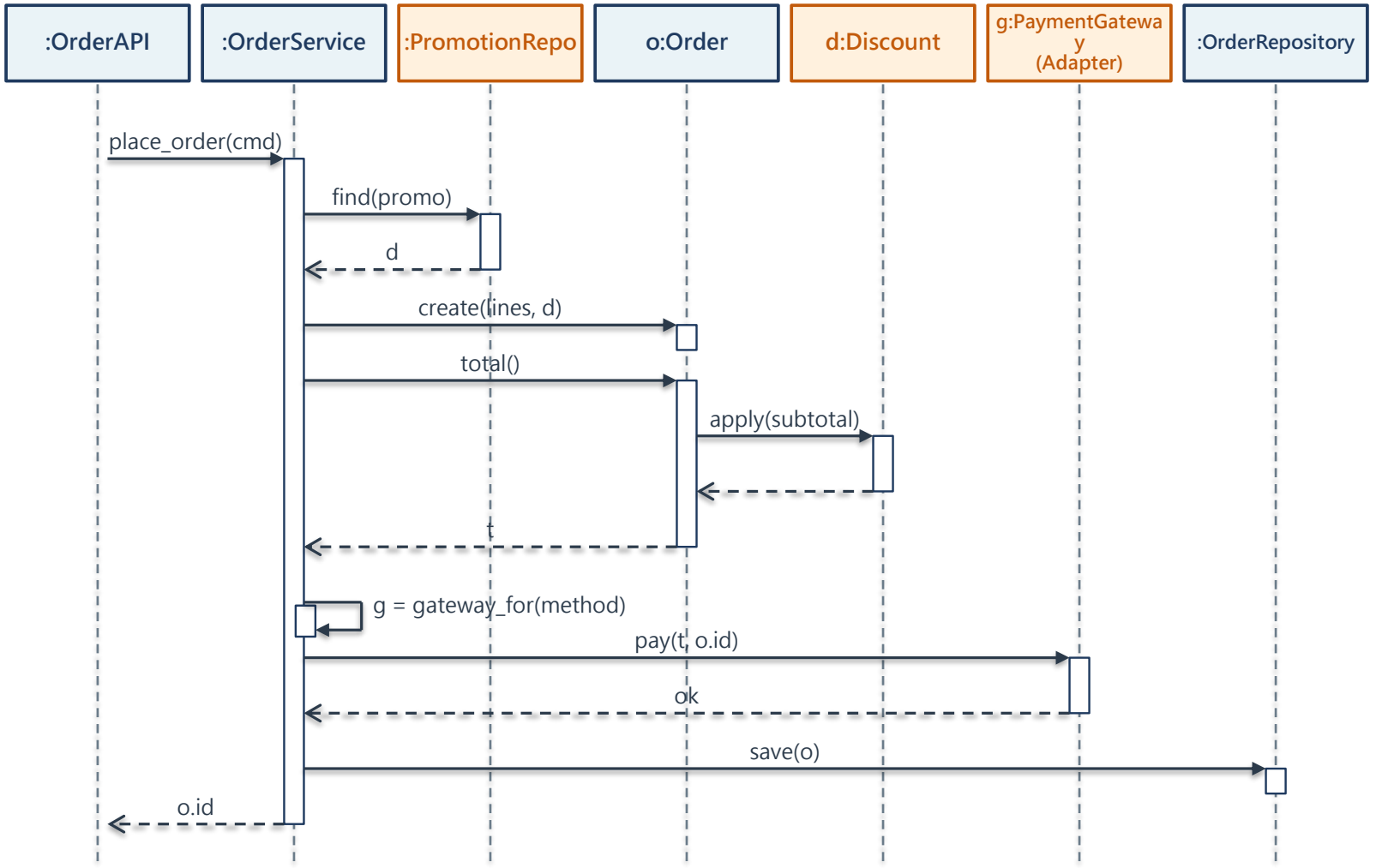
- 資料存取集中，換資料庫只動一層
- 交易邊界清楚

還剩下的問題

- 業務規則的變化（折扣、付款）仍集中在 Service
- Service 直接依賴具體的 Repository 類別

層次 4：Strategy + Factory + Adapter

時效：穩定



- **Strategy**：折扣規則各自一個類別（第 3 部分 UC3）
- **Factory**：gateway_for(method) 用註冊表產生付款 Adapter
- **Adapter**：各家付款 SDK 轉成同一個 pay()
- **剩下的問題**：看不出來：在依賴方向上（下一頁）

層次 4 的變更檢驗

時效：穩定

變更情境	要改的地方	
C1 加折扣	新增一個 Discount 類別	✓
C2 加付款	新增一個 Adapter + 一行註冊	✓
C3 換資料庫	Repository 換掉；但 Order 若是 ORM 模型，欄位改名會牽動領域	△
C4 加入口	新入口呼叫同一個 Service	✓
C5 測試	業務規則可測；但 Order 綁著 ORM，常需要測試資料庫	△

代價：約 12 個類別；規則常變的系統值得

改善了什麼

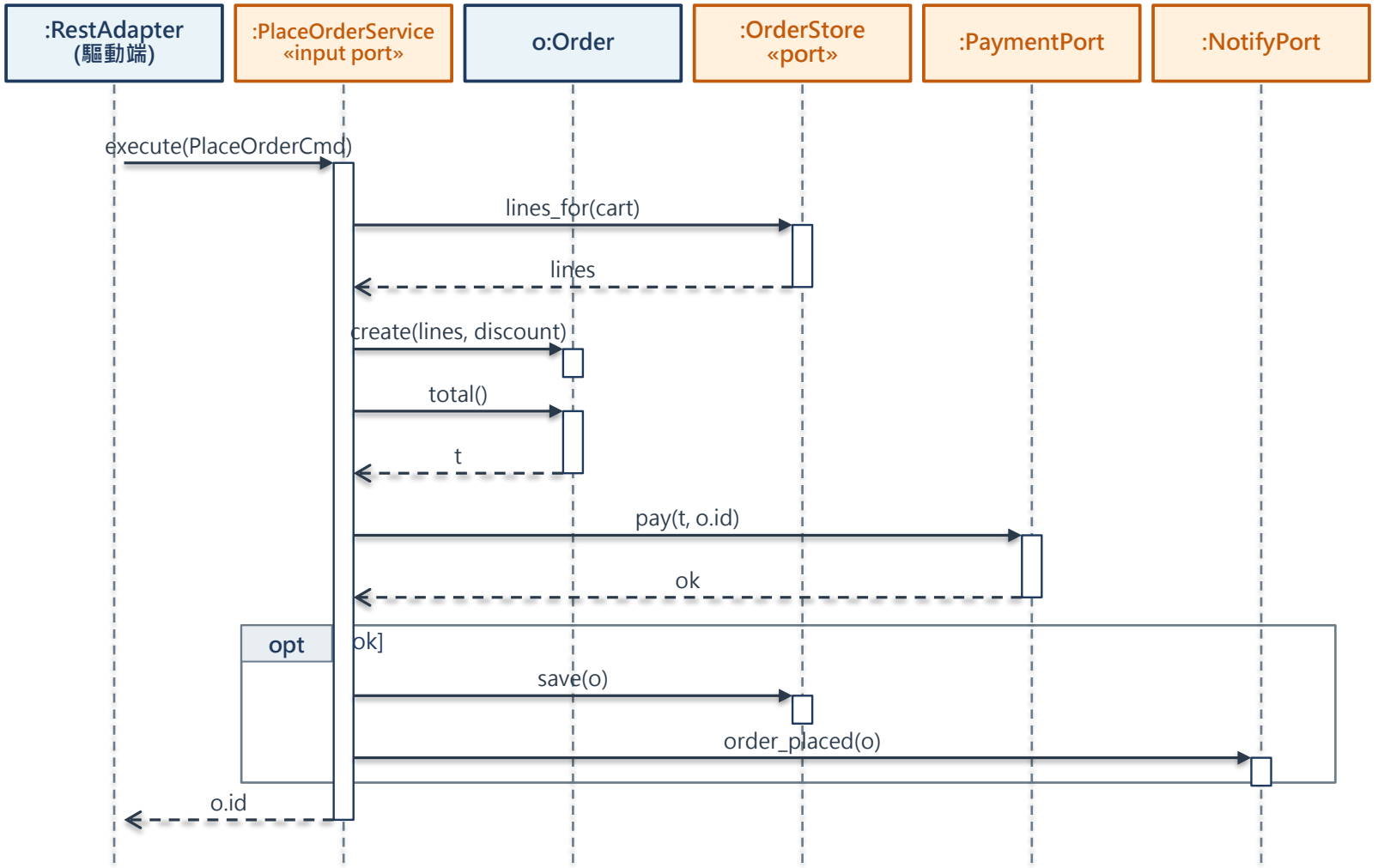
- 業務規則與外部服務都有擴充點
- C1、C2 只需要新增

還剩下的問題

- 領域物件（Order）依賴框架與 ORM
- Service 依賴具體的 Repository 與框架型別：依賴方向仍是「往下指向技術」

層次 5：Hexagonal（Ports & Adapters）——循序圖

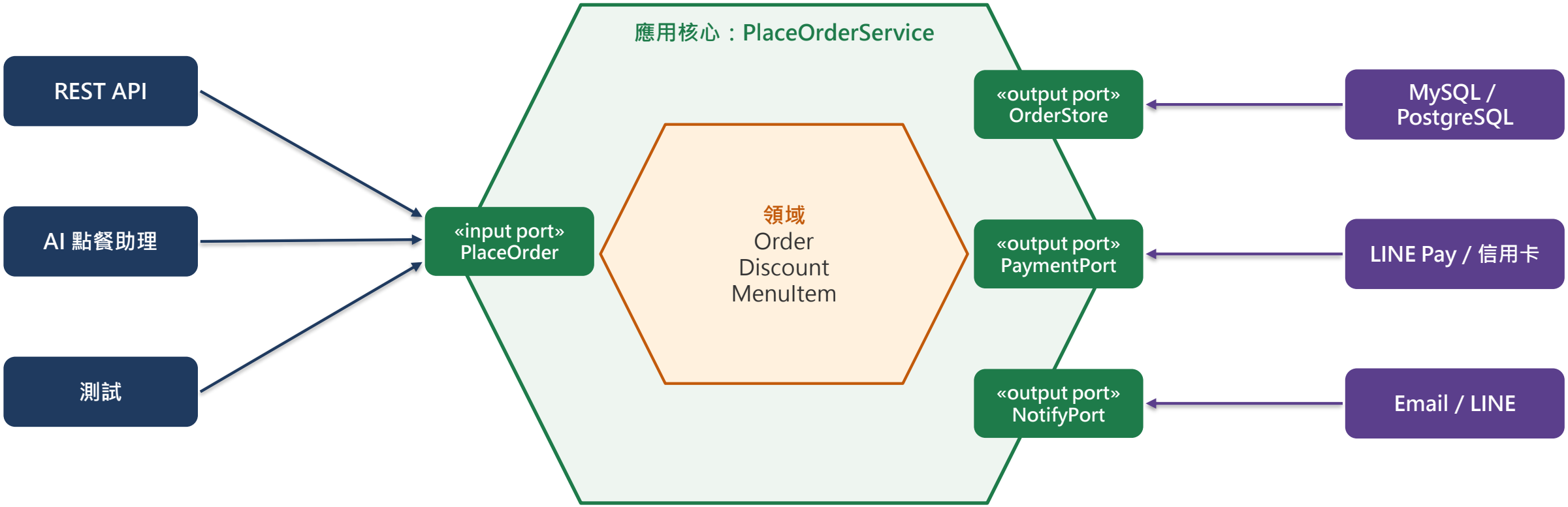
時效：成形中



- **核心只認 port**：OrderStore、PaymentPort、NotifyPort 都是核心自己定義的介面
- **真正的實作在外面**：SqlOrderStore、LinePayAdapter、EmailAdapter 實作這些 port
- **驅動端**：REST、AI 助理、測試都呼叫同一個 input port
- **看起來和層次 4 很像**：差別在箭頭方向：下一頁

層次 5：Hexagonal——依賴全部指向核心

時效：成形中



驅動端 adapter（呼叫核心）

被驅動端 adapter（實作 port）

核心（綠色）不 import 任何框架、資料庫或 SDK；所有箭頭都指向核心——這就是把依賴反轉做到底

層次 5：port 由核心定義，adapter 在外面實作

時效：成形中

核心（core/）

```python
# app/core/ports.py   —— 核心定義，不 import 任何框架
class OrderStore(Protocol):
    def lines_for(self, cart) -> list[Line]: ...
    def save(self, order: Order) -> None: ...

class PaymentPort(Protocol):
    def pay(self, amount: int, order_id: str) -> bool: ...

# app/core/place_order.py
class PlaceOrderService:                    # input port 的實作
    def __init__(self, store: OrderStore,
                 payment: PaymentPort, notify: NotifyPort):
        ...
```

外圈（adapters/）

```python
# app/adapters/sql_store.py —— 外圈：可以用 SQLAlchemy
class SqlOrderStore(OrderStore):
    def __init__(self, session): self.session = session
    def save(self, order):
        self.session.add(OrderRow.from_domain(order))

# app/adapters/linepay.py
class LinePayAdapter(PaymentPort):
    def pay(self, amount, order_id): ...

# app/main.py —— 組裝（composition root）
service = PlaceOrderService(SqlOrderStore(session),
                            LinePayAdapter(sdk),
                            EmailNotifier(smtp))
```

層次 5 的變更檢驗

時效：穩定

變更情境	要改的地方	
C1 加折扣	新增一個 Discount 類別（核心內）	✓
C2 加付款	新增一個 PaymentPort 的 adapter	✓
C3 換資料庫	新寫一個 OrderStore adapter；核心不動	✓
C4 加入口	新增驅動端 adapter（AI 助理、MCP）	✓
C5 測試	核心可用假的 port 完整測試，不需任何框架	✓

代價：約 16 個類別；入口多、外部服務多、要長期維護的系統值得

改善了什麼

- 依賴方向指向核心：技術可以換、核心不受影響
- AI 助理、MCP、LLM 都只是另一個 adapter（第 4、5 部分）

還剩下的問題

- 檔案與介面最多；ORM 模型與領域模型要互相轉換
- 小系統會覺得繁瑣

比較

五層比較：用變更影響取代星等

時效：穩定

層次	C1 折扣	C2 付款	C3 資料庫	C4 入口	C5 測試	類別數
1 單一 Controller	✗	✗	✗	✗	✗	1
2 BCE	✗	✗	✗	✓	△	≈5
3 分層 + Repository	✗	✗	✓	✓	△	≈7
4 Strategy / Factory / Adapter	✓	✓	△	✓	△	≈12
5 Hexagonal	✓	✓	✓	✓	✓	≈16

讀法

每往上一層，就有一種變化被「隔離」到只需新增的程度；代價是類別數增加。好設計 = 把隔離花在真正會變的地方。

比較

什麼時候停在哪一層？

時效：穩定

情況	建議	理由
展示原型、一次性腳本	層次 1 也可以	設計投資回收不了；但要知道它是原型
以 CRUD 為主、規則少	層次 3	資料存取隔離就夠；多數管理系統停在這裡
業務規則常變（折扣、計費）	層次 4	為會變的維度開擴充點（Strategy、Adapter）
多入口、多外部服務、長期維護、有 AI 功能	層次 5	LLM、向量庫、MCP 都會換：讓它們都是 adapter
不確定	從 3 開始，痛了再往上重構	Rule of Three；第 3 部分示範從 1 走到 4 的路

層次不是時代的先後，而是「為哪些變化付出多少結構」的選擇

每一層能寫什麼測試？

時效：穩定

層次	領域單元測試	Service 測試（假物件）	整合測試（真 DB）	端對端（API）
1 單一 Controller	✗	✗	只能整段測	✓
2 BCE	△（Entity 綁 DB）	✗	✓	✓
3 分層 + Repository	△	△（要能注入）	✓（Repository）	✓
4 Strategy…	✓（Discount）	✓	✓	✓
5 Hexagonal	✓	✓（假 port）	✓（每個 adapter）	✓

測試金字塔：越往下越多、越快、越穩定
層次越高，下面那一大塊才寫得出來



層次 5 的單元測試：不需要任何框架

時效：穩定

test_place_order.py

```python
class FakeStore(OrderStore):
    def __init__(self, products): self.products, self.saved = products, []
    def lines_for(self, cart): ...
    def save(self, order): self.saved.append(order)

class FakePayment(PaymentPort):
    def __init__(self, ok): self.ok = ok
    def pay(self, amount, order_id): return self.ok

def test_failed_payment_does_not_save_order():
    store = FakeStore(MENU)
    svc = PlaceOrderService(store, FakePayment(ok=False), FakeNotify())
    with pytest.raises(PaymentFailed):
        svc.execute(PlaceOrderCmd(cart=[("雞腿便當", 2)], method="linepay"))
    assert store.saved == []
```

要點

- **假物件（fake）**：實作同一個 port 的簡單類別；比 mock 好讀
- **測的是規則**：「付款失敗不存訂單」——層次 1 那段 AI 程式正好做錯這件事
- **跑得快**：毫秒級，每次 commit 都能跑
- **整合測試另外寫**：SqlOrderStore 用測試資料庫（例如 SQLite in-memory）
- **API 測試**：FastAPI 的 TestClient 打 RestAdapter

AI

AI 時代的意義：層次名稱是 prompt 的詞彙

時效：成形中

為什麼這一部分在 AI 時代更重要

- **AI 的預設是層次 1**：只說「幫我寫下單」，得到的就是單一 Controller（第 2 部分）
- **層次名稱很精確**：說「用 Hexagonal，核心不得 import 框架」，AI 就知道要產生 port 與 adapter
- **變更情境就是驗收題**：請 AI 回報：C1–C5 各要改哪些檔？
- **AI 功能本身就是 adapter**：LLM、向量庫、MCP 在層次 5 裡只是外圈的 adapter（第 4、5 部分）

hexagonal_prompt.md

```
# 任務：以 Hexagonal 實作「下單」
結構：
  app/core/       領域與 use case；只能用標準函式庫
  app/adapters/   REST、SQL、付款、通知
  app/main.py     組裝
規則：
- core/ 不得 import fastapi、sqlalchemy、
  requests、任何付款或 LLM SDK
- port 由 core/ 定義（Protocol）
- 付款失敗不得儲存訂單
交付：
1. 程式碼與 core/ 的單元測試（用假的 port）
2. 依賴規則的架構測試（見下一頁）
3. 變更情境 C1–C5：各要新增或修改哪些檔
```

把依賴規則寫成測試：AI 違反就紅燈

時效：成形中

test_architecture.py

```python
import ast, pathlib

FORBIDDEN = {"fastapi", "sqlalchemy", "requests",
             "openai", "anthropic", "linepay_sdk"}

def test_core_has_no_infrastructure_imports():
    for f in pathlib.Path("app/core").rglob("*.py"):
        for node in ast.walk(ast.parse(f.read_text())):
            if isinstance(node, ast.Import):
                names = [a.name for a in node.names]
            elif isinstance(node, ast.ImportFrom):
                names = [node.module or ""]
            else:
                continue
            for n in names:
                assert n.split(".")[0] not in FORBIDDEN, f"{f}: {n}"
```

要點

- **架構測試**：把「核心不依賴技術」從口頭約定變成每次都會跑的檢查
- **對 AI 特別有用**：AI 常為了方便直接 import SDK；這個測試會擋下來
- **現成工具**：Python：import-linter；Java：ArchUnit

規格寫在 prompt 裡，AI 可能忘記；規格寫成測試，AI 每次都要過

練習

課堂練習：五層設計 + 變更情境審查

時效：穩定

步驟

- **1**：請 AI 以本頁的 use case 產生層次 3 與層次 5 的 Python（FastAPI）實作
- **2**：請 AI 依程式反畫兩張循序圖（第 2 部分的規格範本）
- **3**：各組自己做 C1（滿千送百）與 C2（街口支付），記錄實際改了哪些檔
- **4**：加上架構測試，故意在 core/ import requests，確認測試會失敗
- **5**：把結果填進「五層比較表」，和本部分的預測對照

討論

- 你預測的 ✓ / △ / ✗ 和實際改動一致嗎？哪裡不一致？為什麼？
- AI 在哪一層最容易「偷偷違反」規則？
- 你們的專題應該停在哪一層？用變更情境說明
- 進階：付款成功但寄信失敗，要怎麼處理？（第 5 部分的 outbox）

第 1 部分：自我檢核

1. 用「變更影響」定義解耦，用「改變的理由」定義內聚，各舉一個下單系統的例子。

2. 層次 3 已經有 Repository，為什麼「換資料庫」仍可能牽動領域？層次 5 怎麼解決？

3. 依賴反轉中，介面應該由哪一方定義？為什麼？

4. 你們的專題適合停在哪一層？用 C1–C5 中至少兩個情境說明。

5. 為什麼「架構測試」對 AI 產生的程式特別有用？

第 2 部分

細部設計變成給 AI 的規格

同一個需求，給 AI 一句話和給 AI 一份設計，得到的是兩種系統

- AI 產生的程式為什麼能跑卻難維護
- 情境表 → 循序圖 → 類別圖：把 Ch9、Ch11 的產出當成規格
- 給 AI 的設計規格範本
- 反畫循序圖：用設計驗收 AI 的程式

實驗：只說一句「幫我寫下單功能」

時效：穩定

order_controller.py（AI 一次產生）

```python
class OrderController:
    def place_order(self, user_id, cart, promo, pay_method, card_no):
        db = mysql.connect("db", "root", "pw")       # 連線寫在流程裡
        total = 0
        for item in cart:
            row = db.query(f"SELECT * FROM product WHERE id={item['id']}")
            if row.stock < item["qty"]:
                return "庫存不足"
            total += row.price * item["qty"]
            if item.get("is_combo"):                  # 套餐特例
                for sub in item["subs"]:
                    total += sub["extra"]
        if promo == "PCT10":                          # 折扣的 if 鏈
            total = total * 0.9
        elif promo == "MINUS50":
            total -= 50
        if pay_method == "credit":                    # 付款的 if 鏈
            ok = CreditGateway().charge(card_no, total)
        elif pay_method == "linepay":
            ok = LinePaySDK().request(amount=total, currency="TWD")
        db.execute(f"INSERT INTO orders VALUES ({user_id}, {total})")
        send_mail(user_id, f"訂單成立，金額 {total}")
        return "OK"
```

它能跑，但.....

- **一個方法做了七件事**：連線、查詢、庫存、計價、套餐、折扣、付款、寄信
- **資料庫細節到處都是**：SQL 字串散在流程中，還有 SQL injection
- **兩條 if 鏈**：每加一種折扣或付款方式，就要改這個方法
- **沒有辦法測試**：連線與外部 API 都寫死在裡面
- **付款失敗也照樣存訂單**：ok 算出來卻沒用到
- **結論**：這不是 AI 能力不夠，是沒有人告訴它責任邊界

AI 產生程式的常見壞味道，對照本單元的原理

時效：穩定

壞味道	在 AI 程式裡的樣子	違反的原理	本單元的對策
臃腫的 Controller	一個方法從頭做到尾	Ch10 Controller 只協調	Expert、Creator
介面層做業務	Flask 路由裡直接算折扣	關注點分離	BCE、Controller
資料庫細節散落	SQL 字串出現在流程中	Ch17 持久化框架	Repository、Bridge
型別分支 if 鏈	if promo == ... elif ...	開放封閉	Strategy、Factory
外部 API 直接呼叫	流程裡直接用 LinePaySDK	低耦合	Adapter
重複的步驟	每個查詢都連線、執行、關閉	不重複	Template Method
巢狀結構的特例	if item.is_combo: for sub ...	一致對待	Composite
反過來：過度設計	只有一種付款也先做 interface + factory	YAGNI	第三次再抽象

為什麼 AI 會這樣寫？

時效：成形中

原因

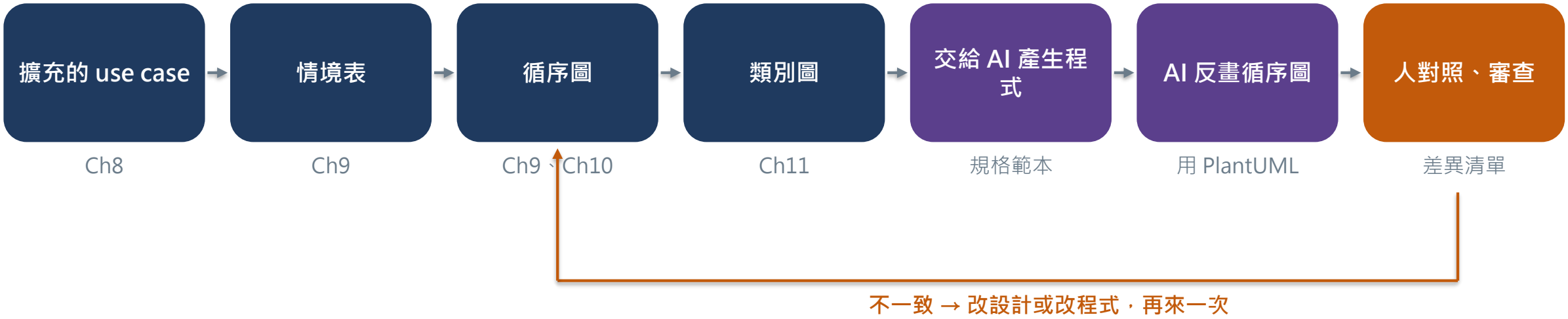
- **預設走最短路徑：**「能跑」是它最容易確認的目標；結構好不好，題目沒說
- **看得到的範圍有限：**只看到這一段需求，看不到明年會加的折扣與付款方式
- **沒有被指派責任邊界：**沒有人說「資料庫只能透過 Repository」
- **訓練資料的平均值：**網路上的範例程式多半是教學用的單一檔案

所以要給它什麼

- **物件與責任：**有哪些物件、每個物件負責什麼（循序圖）
- **介面：**類別、方法、參數、回傳型別（類別圖）
- **限制：**哪些依賴不准出現（例如 Controller 不准直接用 SQL）
- **驗收方式：**測試，以及「把你寫的程式反畫成循序圖」
- **一句話：**設計就是給 AI 的規格，也是驗收 AI 的標準

流程：從 use case 到 AI 產生的程式，再回到設計

時效：成形中



重點

- 前四步就是 Ch8–Ch11 已經學過的產出：差別只在它們現在有了「讀者」——AI
- 後三步是新的：讓 AI 把程式反畫成循序圖，人只要比對兩張圖，就能抓到責任放錯、多出的依賴、漏掉的步驟

第一步：情境表（只寫需要後端處理的步驟）

時效：穩定

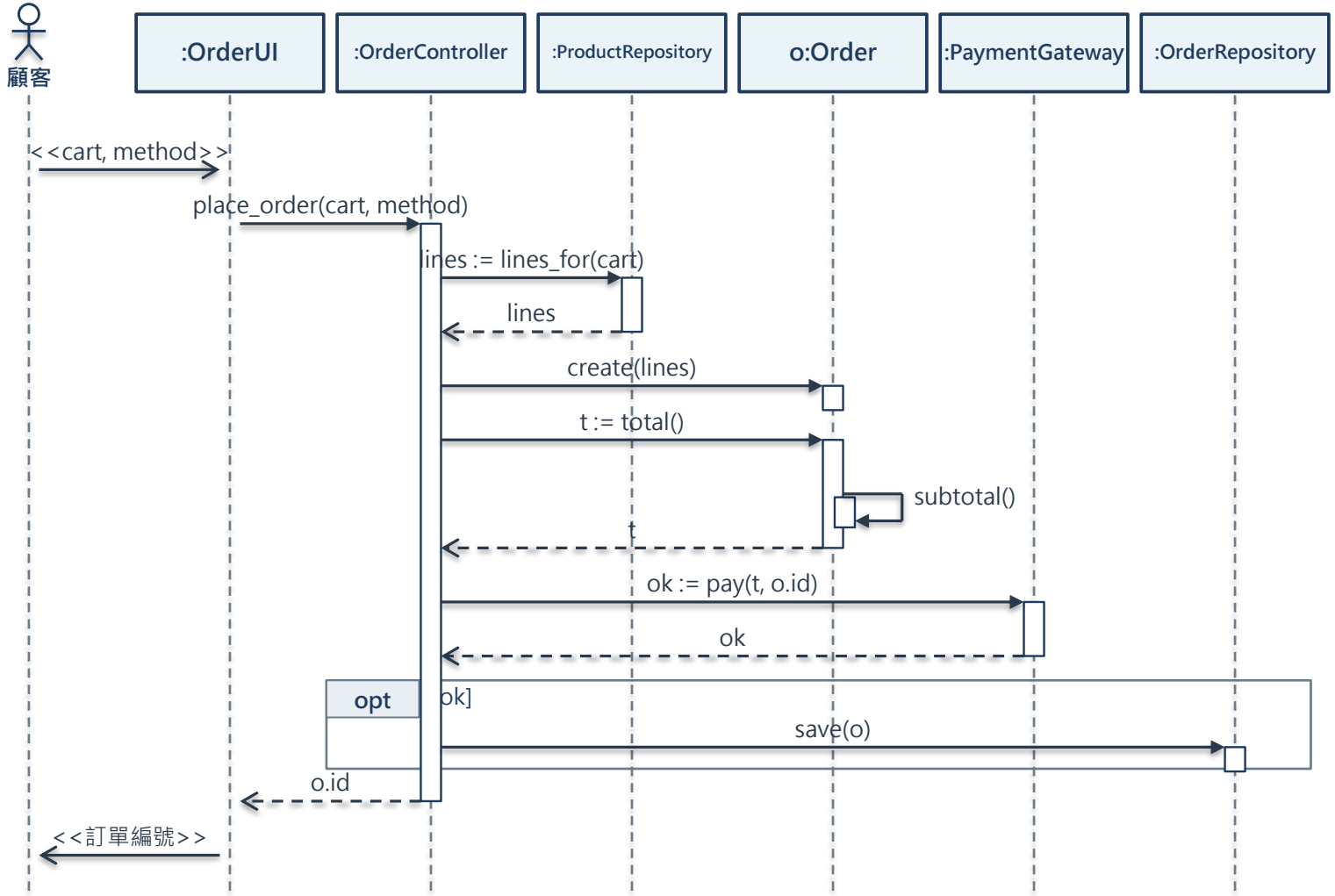
UC1 下新訂單：「顧客按下送出訂單」這一步需要後端處理。每一句：主詞 + 動作 + 其他資料 / 物件 + 受詞（Ch9）

#	主詞	動作	其他資料 / 物件	受詞
1	顧客	送出購物車與付款方式	cart、method	:OrderUI
2	:OrderUI	請求下單	cart、method	:OrderController
3	:OrderController	依商品編號取得訂購明細	id、qty	:ProductRepository
4	:OrderController	建立訂單	lines	o:Order
5	o:Order	計算總金額	—	o:Order
6	:OrderController	請款	total、order id	:PaymentGateway
7	:OrderController	[付款成功] 儲存訂單	o	:OrderRepository
8	:OrderUI	顯示結果	訂單編號或失敗原因	顧客

寫情境表時就在做責任指派：第 5 句由 Order 自己算（Expert），第 4 句由 Controller 建立（Creator）

第二步：循序圖（每個訊息都是一個方法）

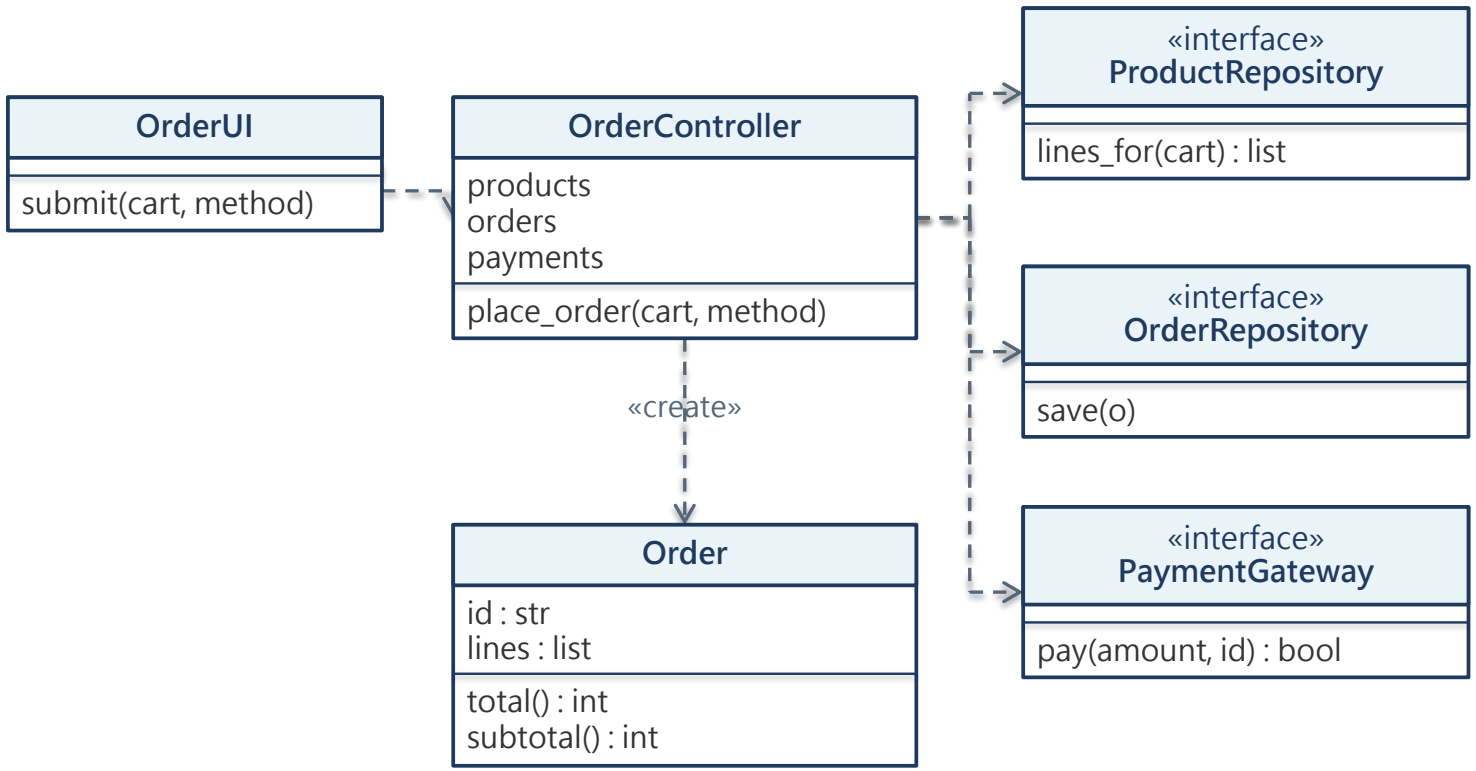
時效：穩定



- **Controller 只協調**：它不算錢、不碰 SQL
- **Expert**：total() 由 Order 自己算
- **Creator**：Controller 有建立 Order 所需的明細
- **外部系統**：只透過 PaymentGateway
- **opt 框**：付款成功才存訂單：一句話的修正

第三步：類別圖（從循序圖萃取，Ch11 的四個步驟）

時效：穩定



萃取規則（Ch11）

- **類別**：循序圖上收送訊息的物件、參數與回傳型別
- **方法**：每一個「射進來」的訊息
- **屬性**：從 get / set 與領域模型找
- **關係**：有呼叫就有相依（虛線箭頭）
- **新增的一條**：三個 Repository / Gateway 標成 interface：這是給 AI 的「不准越界」宣告

第四步：給 AI 的設計規格範本

時效：成形中

spec_prompt.md

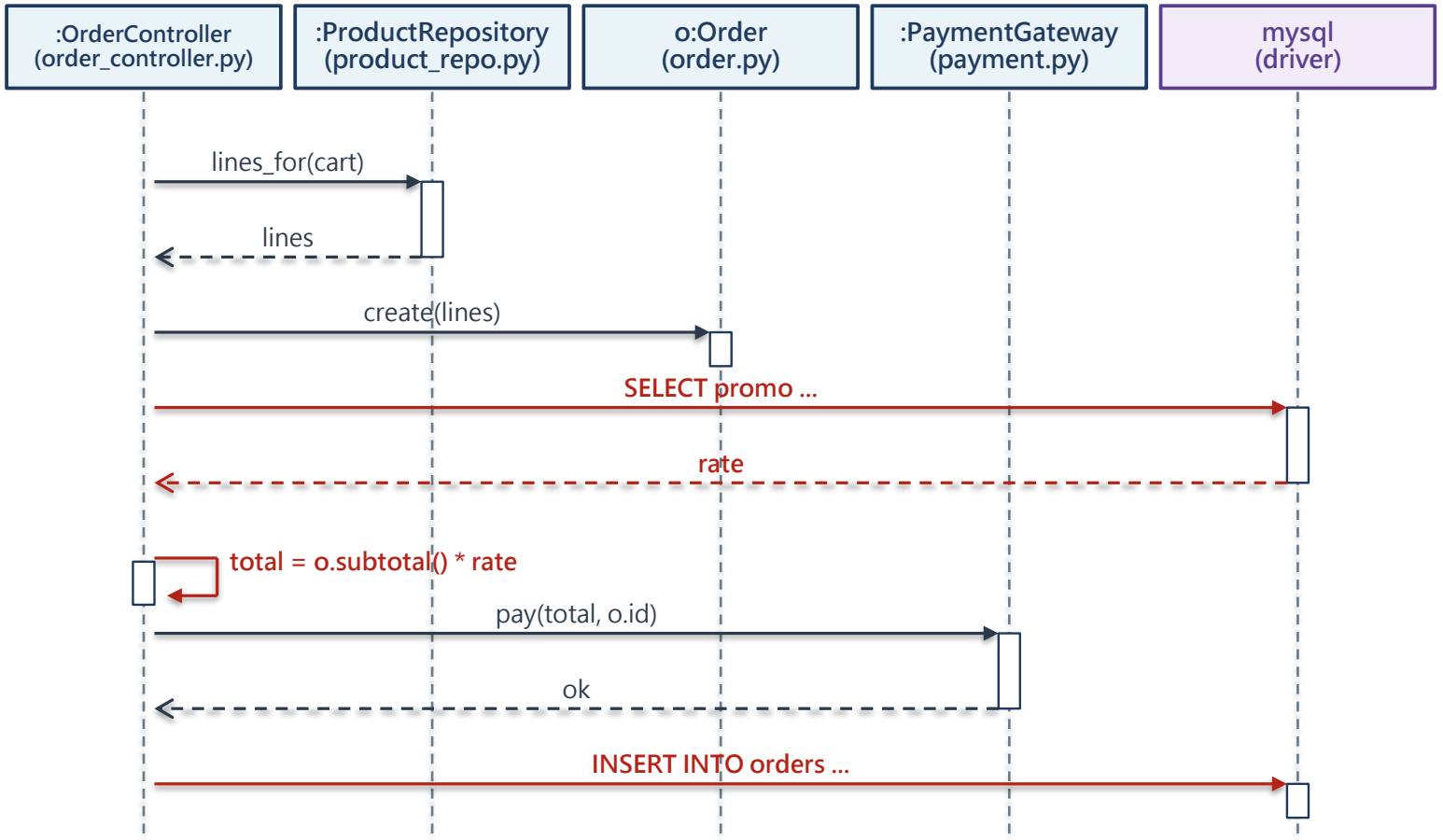
```
# 任務：實作 UC1 下新訂單（Python 3.12）
# 1. 附件：情境表、循序圖（PlantUML）、類別圖
# 2. 只能建立類別圖上列出的類別與方法；
#    需要新增時，先列出來說明理由，不要直接寫
# 3. 每個類別一個檔案；檔名 = 類別名稱的 snake_case
# 4. 循序圖上的每一個訊息，都要對應一個方法呼叫
# 5. 限制：
#    - OrderController 不得出現 SQL、不得 import SDK
#    - 金額只能由 Order 計算
#    - 外部服務一律透過建構子注入（方便測試）
# 6. 交付：
#    a. 程式碼
#    b. pytest 測試（用假的 Repository 與 Gateway）
#    c. 依你寫出的程式「反畫」PlantUML 循序圖，
#       物件名稱標成「類別（檔名.py）」
#    d. 與附件循序圖不一致之處的清單
```

每一條在防什麼

- **第 2 條**：防止 AI 自己發明結構
- **第 3、6c 條**：去年 prompt 的規則：循序圖要標出類別與檔名，才對得回程式
- **第 4 條**：設計與程式一一對應，審查才有依據
- **第 5 條**：責任邊界寫成可以檢查的句子
- **第 6b 條**：注入讓測試可以不連資料庫
- **第 6d 條**：讓 AI 自己先報告偏離之處

第五步：讓 AI 反畫循序圖，人只比對差異

時效：成形中



AI 反畫出來的圖，紅色是與設計不一致：

- Controller 直接查資料庫拿折扣
- 金額由 Controller 算，不是 Order
- 存訂單繞過 OrderRepository

審查只要問三句：

- 多出了哪些物件或依賴？
- 哪個訊息的發送者換人了？
- 少了哪個步驟？

反畫審查清單（比對兩張循序圖）

時效：成形中

檢查	在圖上看什麼	常見原因
多出的參與者	設計沒有的 lifeline（mysql、requests、SDK）	越過抽象直接呼叫
發送者換人	原本由 Order 發出的訊息，變成 Controller 發出	責任被搬到 Controller（臃腫）
少掉的步驟	設計上有的訊息，程式裡沒有	例外路徑被省略（付款失敗）
框不見了	alt / opt / loop 消失	條件寫進了別的方法，或根本沒處理
觸發順序錯	先呼叫、後收到使用者輸入（Ch9 常見錯誤）	事件順序沒想清楚
activation 沒結束	一個方法包住後面所有呼叫	流程與資料存取混在一起

這份清單就是 Ch9「常見錯誤」與 Ch10「臃腫的 Controller」換一個方向使用：用來審 AI，也用來審自己

規格要寫到多細？

時效：穩定

一定要寫

- 有後端處理的步驟（nontrivial step）
- 物件與責任分配
- 介面：方法名、參數、回傳
- 不准出現的依賴
- 驗收的測試

可以交給 AI

- 方法內部的演算法細節
- 例外處理的實作方式（Ch9：留給 coding）
- 命名的小調整
- 格式與文件字串

依風險決定

- 金錢、個資、權限相關：寫細
- 會頻繁變動的地方（折扣、付款）：寫出擴充點
- 一次性、低風險：一句話也可以
- Ch9 的 Agile 原則仍然適用：Keep it simple; good enough is enough

第 2 部分：自我檢核

1. 指出「AI 一次產生」那段程式中的三個壞味道，各對應本單元哪一條原理。

2. 情境表中的哪一句，決定了 Expert pattern 的使用？

3. 為什麼規格範本要求「依程式反畫循序圖」？只看程式碼審查有什麼困難？

4. 在規格中寫「OrderController 不得出現 SQL」，對應 Ch17 的哪一個設計？

5. 舉一個你認為「一句話給 AI 就夠了」的情境，並說明理由。

第 3 部分

用 AI 重構下單系統

取代去年的「Refactoring & Design Patterns」：每一步都有壞味道、安全網、手法與循序圖

- 第 1 部分是「五種設計長什麼樣」；這一部分是「怎麼從層次 1 一步一步走到層次 4」
- 重構是什麼、不是什麼；壞味道與手法對照表
- 安全網：先把現在的行為釘住
- UC1 Controller + Expert + Creator　UC2 Command　UC3 Strategy
- UC4 Composite　UC5 Factory（註冊表）+ Adapter
- 改一個需求要動幾個檔？　AI 的重構節奏與過度設計警訊

說明

去年的補充少了什麼？今年怎麼改

時效：穩定

去年的版本	今年的版本
只給每一步的最終程式	每一步：壞味道 → 手法 → 改完的程式 → 循序圖前後對照
沒有測試	重構前先寫特徵測試（characterization test），每一步都要綠燈
UC2 的 Command 只有 print	排隊、主管核准、取消 = 撤銷：Command 的價值才出得來
UC3 用 if strategy: 判斷有沒有折扣	預設的 NoDiscount 策略，把 if 消掉；誰選策略也交代清楚
UC5 標成「Factory + Bridge」	其實是 Factory + 多型；改成註冊表 Factory + Adapter，Bridge 留到第 4、5 部分
Expert 只出現在標題	「算金額」從 Controller 搬到 Order，並畫出前後循序圖
AI 只在最後一頁的 prompt	每一步都說明 AI 做什麼、人守什麼；prompt 擴充成規格範本（第 2 部分）

說明

重構是什麼、不是什麼

時效：穩定

重構（Refactoring）

- **定義**：不改變外部行為的前提下，改善內部結構
- **前提**：有測試證明「行為沒變」
- **節奏**：一次一個手法、每步都可以跑、每步都 commit
- **目的**：讓下一個需求好加：降低耦合、提高內聚

不是重構

- **順便加功能**：一邊重構一邊加折扣：出錯時不知道是哪一個造成的
- **整個重寫**：「AI 幫我全部重寫一遍」：沒有任何保證行為一樣
- **只是改名排版**：那是整理，不會改變相依關係
- **為了用 pattern 而用**：沒有痛點的抽象，只會讓程式更難讀

說明

壞味道 → 重構手法 → pattern 對照表

時效：穩定

壞味道	徵兆	重構手法	常落到的 pattern
臃腫的方法 / Controller	一個方法做很多件事	Extract Method、Move Method	Controller、Expert
資料細節散落	SQL、連線字串到處都是	Extract Class + 注入	Repository、Bridge
型別分支	if type == ... elif ...	Replace Conditional with Polymorphism	Strategy
建立物件的 if 鏈	新增一種就要改工廠	以註冊表取代分支	Factory
外部 API 形狀不一	每家 SDK 參數不同	Introduce Interface	Adapter
操作要排隊、核准、撤銷	狀態欄位與 if 散在各處	Replace Method with Method Object	Command
巢狀結構的特例	if is_combo: for sub ...	Replace Conditional with Polymorphism	Composite
重複的步驟骨架	每個查詢都連線、執行、關閉	Form Template Method	Template Method

安全網：先把「現在的行為」釘住

時效：穩定

step0_inject.py

```python
# 第 0 步：只做一件事——把外部依賴改成從外面傳入
class OrderController:
    def __init__(self, db, credit, linepay, mailer):
        self.db, self.credit = db, credit
        self.linepay, self.mailer = linepay, mailer
```

test_characterization.py

```python
def test_place_order_current_behavior():
    db = FakeDB(products={1: (120, 10), 2: (60, 10)})
    pay = FakeCredit()
    c = OrderController(db, pay, FakeLinePay(), FakeMailer())
    c.place_order("u1", [{"id": 1, "qty": 2}, {"id": 2, "qty": 1}],
                  "PCT10", "credit", "4111")
    assert pay.charged == 270      # (240 + 60) × 0.9
```

為什麼先做這兩件事

- **特徵測試**：不問「對不對」，只記錄「現在是什麼」。之後每一步都跑它
- **第 0 步只注入**：沒有注入就寫不了測試；注入本身是最小、最安全的改動
- **AI 很會寫這種測試**：請它「列出這個方法所有分支，各寫一個測試案例」
- **人要確認**：測試案例真的涵蓋了折扣、套餐、兩種付款與庫存不足
- **發現錯誤怎麼辦**：付款失敗也存訂單——先記下來，重構完再修，不要混在一起

UC1 下新訂單：Controller 什麼都自己來

時效：穩定

重構前

```
class OrderController:
    def place_order(self, user_id, cart, promo, method, card):
        total = 0
        for item in cart:
            row = self.db.query("SELECT * FROM product WHERE id=?",
                                item["id"])
            if row.stock < item["qty"]:
                return "庫存不足"
            total += row.price * item["qty"]
        # ……折扣、付款、存檔、寄信（下一頁以後處理）
        self.db.execute("INSERT INTO orders ...", user_id, total)
        return "OK"
```

聞到什麼味道

- **Controller 在算錢**：價格 × 數量的資訊在商品與訂單身上，卻由 Controller 計算（違反 Expert）
- **Controller 在寫 SQL**：資料庫細節與流程混在一起（違反 Ch17 的關注點分離）
- **沒有 Order 物件**：訂單只是一個 total 數字，之後的折扣、套餐都沒有地方放

UC1 重構後：Controller 協調、Order 算錢、Repository 存取

時效：穩定

重構後

```python
class Order:                                    # Expert
    def __init__(self, lines):
        self.lines = lines
    def subtotal(self):
        return sum(l.price * l.qty for l in self.lines)

class OrderController:                          # Controller
    def __init__(self, products, orders):
        self.products, self.orders = products, orders
    def place_order(self, user_id, cart):
        lines = self.products.lines_for(cart)   # 庫存檢查也在這裡
        order = Order(lines)                    # Creator
        self.orders.save(order)
        return order.id
```

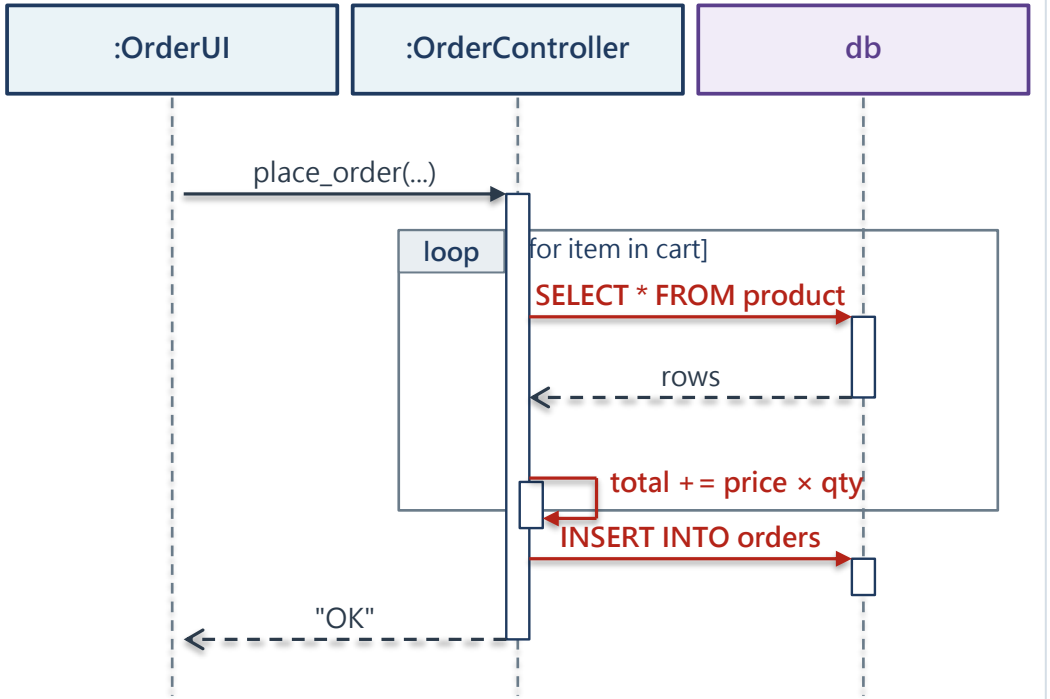
改了什麼

- **Extract Class：Order**：金額由擁有明細的 Order 計算
- **Move Method：subtotal**：從 Controller 搬到 Order
- **Extract Class：Repository**：SQL 全部進 ProductRepository、OrderRepository
- **Creator**：Controller 手上有明細，所以由它建立 Order
- **測試**：特徵測試仍然綠燈，再加 Order 的單元測試

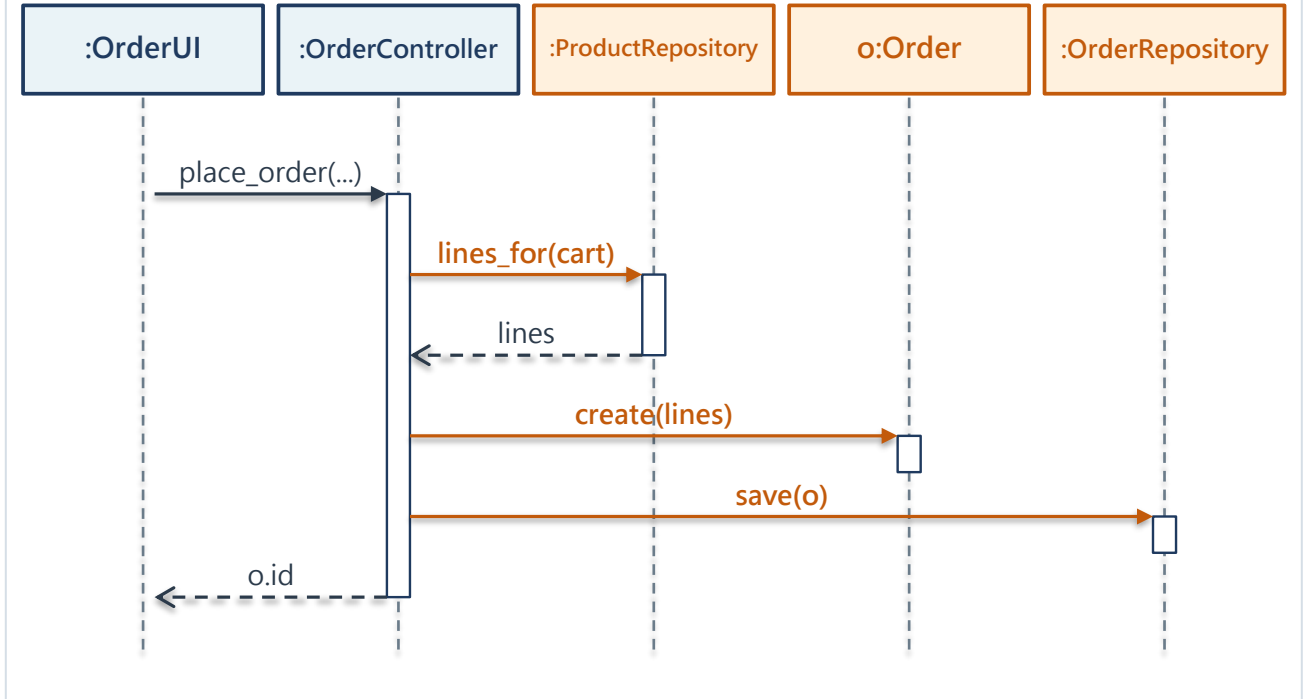
UC1 循序圖前後對照

時效：穩定

✗ 重構前：Controller 的 activation 包住一切



✓ 重構後：每個物件只做自己的事



看圖就知道責任分配：重構前所有箭頭都從 Controller 出發；重構後，Controller 只發出「請求」，計算與存取由專家負責

UC1：AI 做什麼、人守什麼

時效：成形中

AI 擅長做的

- 依指令做 Extract Class、Move Method，速度很快
- 把所有 SQL 收進 Repository，並產生介面
- 補上 Order、Repository 的單元測試
- 反畫重構前後兩張循序圖給你比對

人要守住的

- 決定「誰是 Expert」：金額該由 Order 還是 Product 算？
- 確認庫存檢查放對地方（Repository 只查，判斷規則屬於領域）
- 每一步跑特徵測試，綠燈才 commit
- 拒絕「順便」：AI 常會趁機改掉付款或折扣，要它停下來

UC2 新需求：大額訂單要核准、取消要回補庫存

時效：穩定

重構前

```
class OrderController:
    def place_order(self, user_id, cart):
        order = Order(self.products.lines_for(cart))
        if order.subtotal() > 50000:           # 大額 B2B 訂單
            order.status = "PENDING"            # 等主管核准
            self.orders.save(order)
            return order.id
        self.stock.reserve(order.lines)
        order.status = "PLACED"
        self.orders.save(order)
        return order.id
    def approve(self, order_id):               # 複製一份下單邏輯
        order = self.orders.get(order_id)
        self.stock.reserve(order.lines)
        order.status = "PLACED"
        self.orders.save(order)
    def cancel(self, order_id):                # 自己寫一份反向操作
        order = self.orders.get(order_id)
        if order.status == "PLACED":
            self.stock.release(order.lines)
        order.status = "CANCELLED"
        self.orders.save(order)
```

聞到什麼味道

- **同一個操作寫了兩次：**place_order 與 approve 都在保留庫存、改狀態
- **反向操作散在別處：**「取消」要記得回補庫存，但它和下單分開寫，容易漏
- **狀態字串與 if 到處都是：**PENDING、PLACED、CANCELLED 散在三個方法
- **接下來的需求：**核准紀錄、批次執行、撤銷……都要再改 Controller

UC2 重構後：把「下單」變成一個可以排隊與撤銷的物件

時效：穩定

重構後

```
class OrderCommand:                          # Command 介面
    def execute(self): ...
    def undo(self): ...

class PlaceOrder(OrderCommand):
    def __init__(self, order, stock, orders):
        self.order, self.stock, self.orders = order, stock, orders
    def execute(self):
        self.stock.reserve(self.order.lines)
        self.orders.save(self.order.placed())
    def undo(self):                                # 取消 = 撤銷
        self.stock.release(self.order.lines)
        self.orders.save(self.order.cancelled())

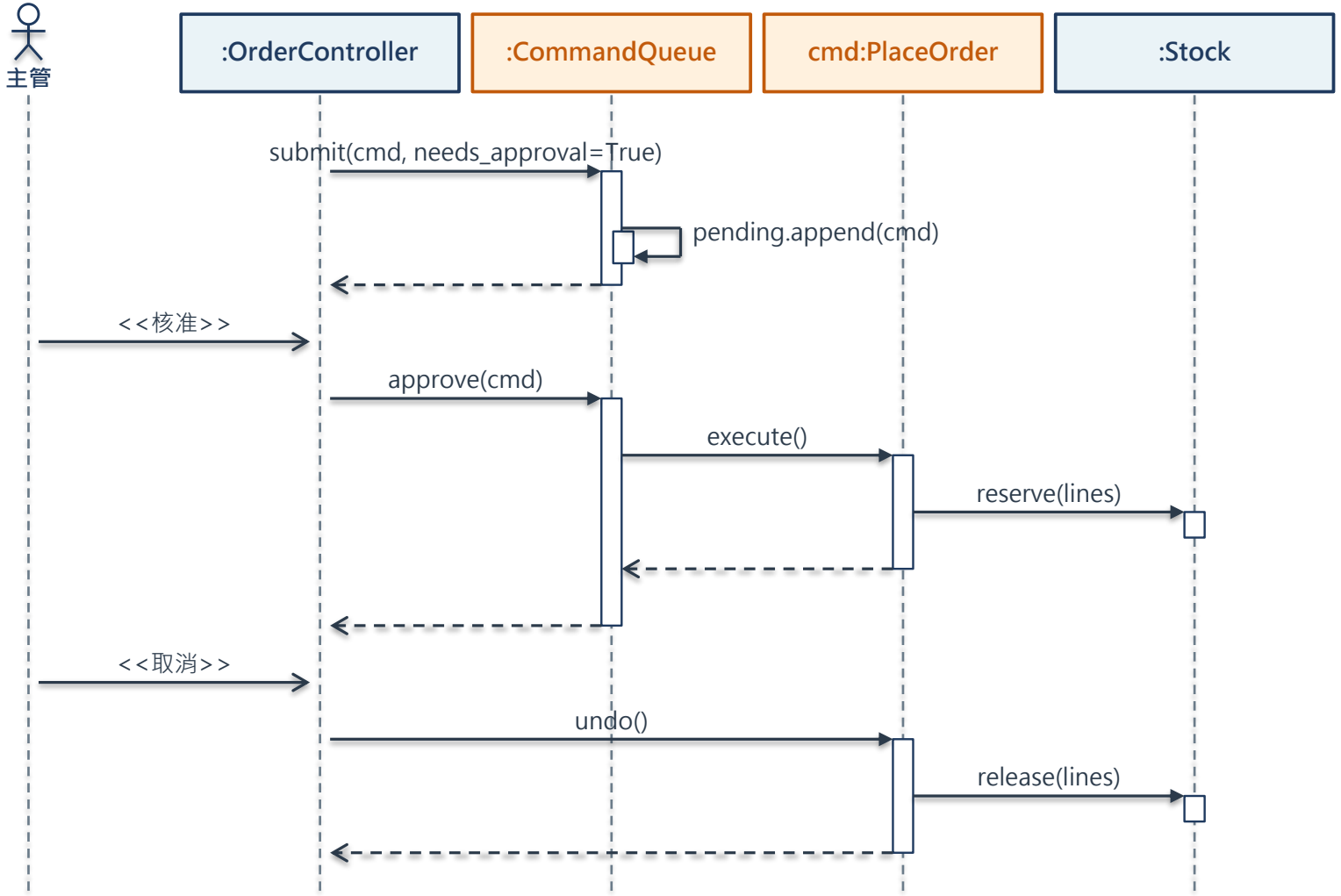
class CommandQueue:
    def __init__(self):
        self.pending, self.history = [], []
    def submit(self, cmd, needs_approval):
        if needs_approval: self.pending.append(cmd)
        else: self.run(cmd)
    def approve(self, cmd):
        self.pending.remove(cmd); self.run(cmd)
    def run(self, cmd):
        cmd.execute(); self.history.append(cmd)
```

改了什麼

- **Command**：一次「下單」就是一個物件：execute 與 undo 寫在一起，不會漏
- **CommandQueue**：需要核准就先排隊；核准後執行；執行過的留在 history
- **Controller 變得很薄**：只決定 needs_approval（例如金額 > 5 萬）
- **呼應 Ch17**：Command 支援 undo / redo、排隊、稍後執行
- **第 4 部分會再用到**：AI 助理要下單前，也要「顧客確認」：同一個佇列

UC2 循序圖：排隊 → 核准 → 執行 → 撤銷

時效：穩定



- **排隊與執行分開**：submit 時不執行；approve 時才 execute
- **撤銷就在旁邊**：undo 由同一個物件負責，知道當初保留了什麼
- **之後要加功能**：核准紀錄、批次核准、重做，都只動 CommandQueue

UC3 折扣：每加一種就多一個 elif

時效：穩定

重構前

```
class OrderController:
    def place_order(self, user_id, cart, promo=None):
        order = Order(self.products.lines_for(cart))
        total = order.subtotal()
        if promo == "PCT10":
            total = round(total * 0.9)
        elif promo == "MINUS50":
            total = max(0, total - 50)
        elif promo == "VIP":
            total = round(total * 0.85)
        # 下個月：滿千送百、第二件半價……
        ...
```

聞到什麼味道

- **型別分支**：promo 的每一種值都是一段 elif
- **Controller 又在算錢**：折扣後金額不在 Order 裡
- **改一個要測全部**：加「滿千送百」要改這個方法，所有折扣都要重測
- **去年的寫法也有問題**：if strategy: 判斷「有沒有折扣」——用預設策略就可以消掉

UC3 重構後：Strategy + 預設策略

時效：穩定

重構後

```
class Discount:                          # Strategy 介面
    def apply(self, amount):
        return amount
class NoDiscount(Discount): pass         # 預設：不用再寫 if
class Percent(Discount):
    def __init__(self, pct): self.pct = pct
    def apply(self, amount):
        return round(amount * (100 - self.pct) / 100)
class Minus(Discount):
    def __init__(self, off): self.off = off
    def apply(self, amount):
        return max(0, amount - self.off)

class Order:
    def __init__(self, lines, discount=NoDiscount()):
        self.lines, self.discount = lines, discount
    def total(self):
        return self.discount.apply(self.subtotal())

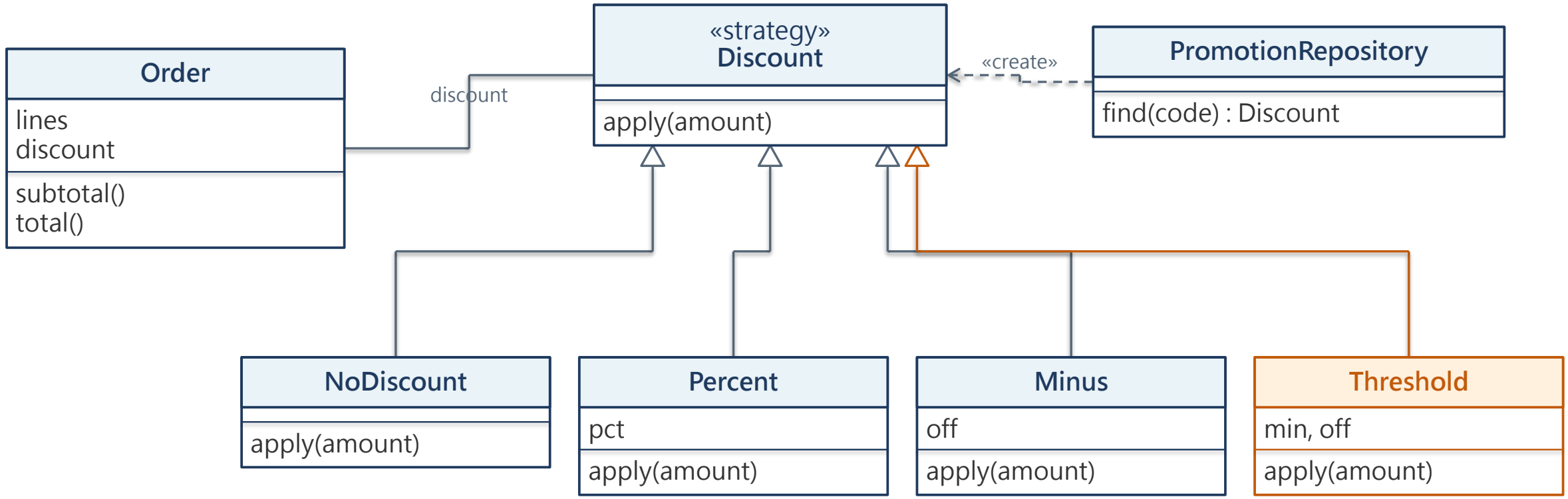
# 誰來選策略？由促銷代碼查表，不寫在 Controller
discount = self.promotions.find(promo)    # 找不到就回 NoDiscount()
```

改了什麼

- **Replace Conditional with Polymorphism**：每一種折扣一個類別
- **NoDiscount**：「沒有折扣」也是一種策略：Order 永遠可以呼叫 apply
- **Expert**：折扣後金額由 Order.total() 負責
- **誰選策略**：PromotionRepository 依代碼回傳策略（資料可以放在資料庫）
- **加「滿千送百」**：新增一個類別，其他程式都不動

UC3 類別圖：新增「滿千送百」只多一個方塊

時效：穩定



橘色是新需求：Threshold（滿 min 元折 off 元）。Order、Controller、其他折扣都不用改——這就是開放封閉

UC4 套餐：套餐裡還有套餐

時效：穩定

重構前

```
def subtotal(self):
    total = 0
    for line in self.lines:
        if line.is_combo:
            combo_price = 0
            for sub in line.subs:
                if sub.is_combo:                # 套餐裡的小套餐
                    for subsub in sub.subs:
                        combo_price += subsub.price
                    combo_price -= sub.saving
                else:
                    combo_price += sub.price
            total += (combo_price - line.saving) * line.qty
        else:
            total += line.price * line.qty
    return total
```

聞到什麼味道

- **巢狀的 if 與 for**：每多一層套餐，就多一層迴圈
- **同一件事寫了三次**：「算價錢」在單品、套餐、小套餐各寫一次
- **呼叫端要知道結構**：Order 必須知道套餐怎麼組成
- **Ch16 的同一個問題**：狀態圖編輯器的「狀態裡有狀態圖」

UC4 重構後：Composite，單品與套餐一視同仁

時效：穩定

重構後

```python
class MenuItem:                          # Component
    def price(self): ...

class Single(MenuItem):                  # Leaf
    def __init__(self, name, p):
        self.name, self.p = name, p
    def price(self):
        return self.p

class Combo(MenuItem):                   # Composite：可以再放 Combo
    def __init__(self, name, parts, saving):
        self.name, self.parts, self.saving = name, parts, saving
    def price(self):
        return sum(x.price() for x in self.parts) - self.saving

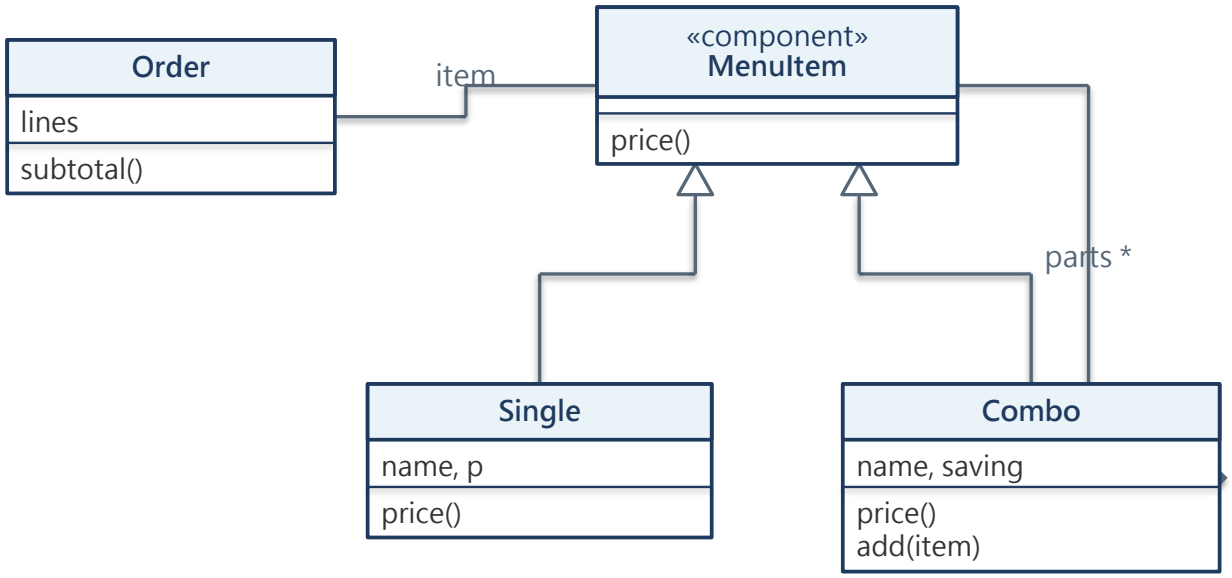
class Order:
    def subtotal(self):
        return sum(l.item.price() * l.qty for l in self.lines)
```

改了什麼

- **Composite**：套餐與單品實作同一個介面 price()
- **遞迴**：Combo 把工作交給它的每一個部分，自然支援任意層
- **Order 變單純**：不必知道品項是單品還是套餐
- **Iterator**：若要列印明細，可以為 Combo 提供逐項走訪，不暴露 parts 的儲存方式
- **第 4 部分會再用到**：AI 產生的「點餐計畫」也是一棵樹

UC4 類別圖與一個實例

時效：穩定



實例：家庭分享餐（一棵樹）

- 家庭分享餐　−80
  - 炸雞桶（8 塊）　399
  - 兒童餐　−20
    - 小漢堡　59
    - 小薯　35
    - 玩具　30
  - 大可樂 × 2　90

price() = 399 + (59+35+30−20) + 90 − 80 = 513

UC5 付款：每家 API 長得不一樣，工廠也在 if

時效：穩定

重構前

```
class PaymentFactory:                          # 去年的版本
    @staticmethod
    def get_processor(method):
        if method == "credit":
            return CreditCardPayment()
        elif method == "linepay":
            return LinePayPayment()
        else:
            raise ValueError("Unsupported")

# 而兩家 SDK 的介面完全不同：
gateway.charge(cents=27000, ref="A1")        # 信用卡：分為單位
linepay.request(amount=270, currency="TWD",  # LINE Pay
                orderId="A1")
```

聞到什麼味道

- **工廠仍是 if 鏈**：新增「街口支付」還是要改 PaymentFactory
- **外部 API 形狀不一**：參數、單位、回傳格式各不相同
- **去年標成 Bridge**：這裡只有一個維度在變化（付款方式），實際是 Factory + 多型，不是 Bridge
- **真正的需求**：「把別人的介面轉成我們要的介面」——這是 Adapter

UC5 重構後：Adapter + 註冊表 Factory

時效：穩定

重構後

```python
class PaymentGateway:                          # 我們要的介面
    def pay(self, amount, order_id) -> bool: ...

class LinePayAdapter(PaymentGateway):          # Adapter
    def __init__(self, sdk): self.sdk = sdk
    def pay(self, amount, order_id):
        r = self.sdk.request(amount=amount, currency="TWD",
                             orderId=order_id)
        return r["returnCode"] == "0000"

class CreditCardAdapter(PaymentGateway):       # Adapter
    def __init__(self, gw): self.gw = gw
    def pay(self, amount, order_id):
        return self.gw.charge(cents=amount * 100,
                              ref=order_id).approved

GATEWAYS = {}                                  # Factory：註冊表
def register(name, maker): GATEWAYS[name] = maker
def gateway_for(name): return GATEWAYS[name]()

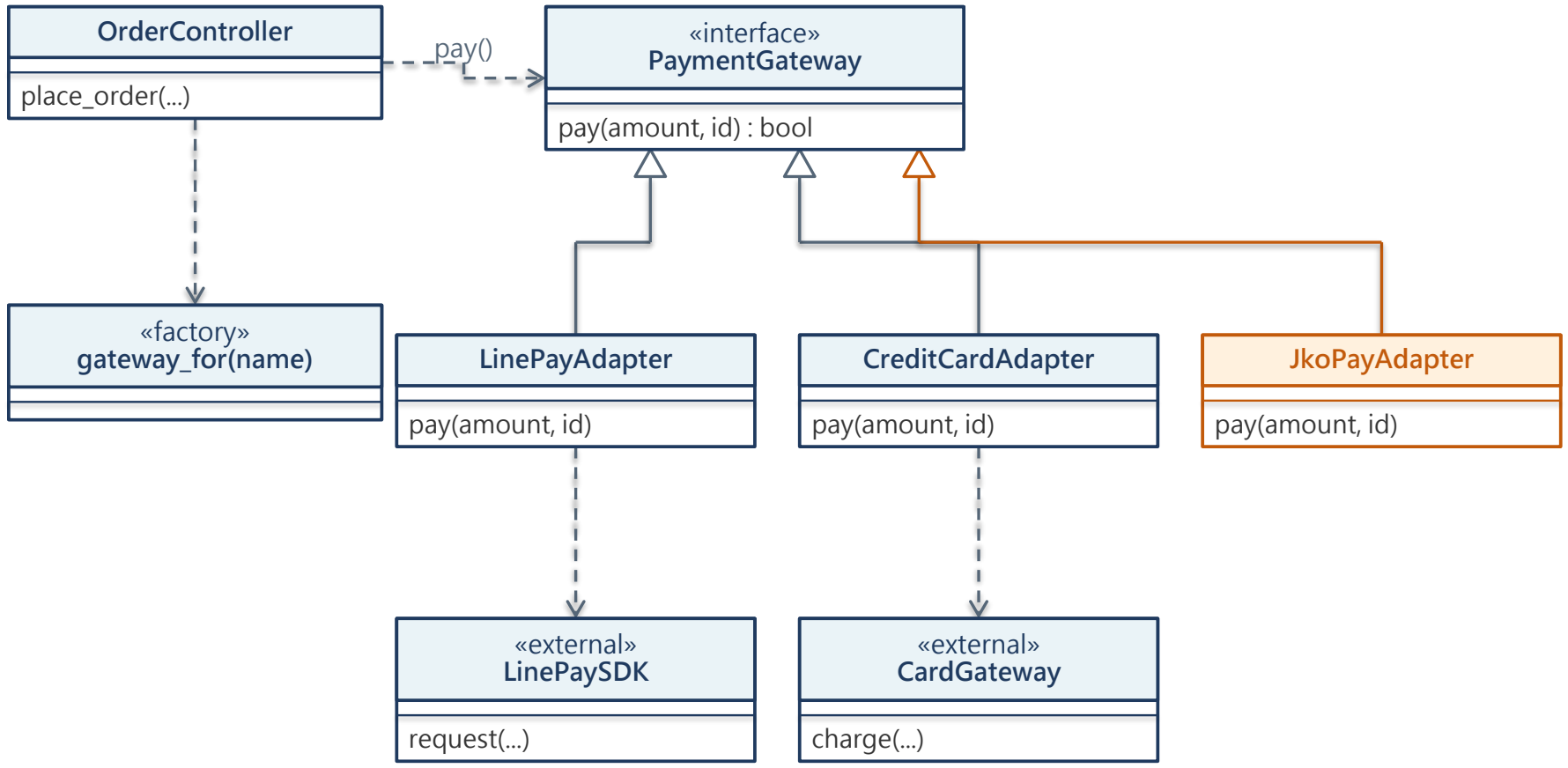
register("linepay", lambda: LinePayAdapter(LinePaySDK()))
register("credit",  lambda: CreditCardAdapter(Gateway()))
```

改了什麼

- **Adapter**：每家 SDK 包一層，對內一律 pay(amount, order_id) → bool
- **單位轉換在 Adapter 裡**：分與元、回傳碼的判斷，不會漏到流程中
- **註冊表 Factory**：新增付款方式 = 新增一個 Adapter + 一行 register
- **測試**：register("fake", ...) 就能換成假的閘道
- **Bridge 是什麼**：兩個維度各自變化時才用（第 4 部分：助理種類 × 模型供應商）

UC5 類別圖：新增「街口支付」只多一個 Adapter

時效：穩定



Adapter 擋住外部 API 的形狀；註冊表擋住「有哪幾種」。兩者合起來，Controller 對付款方式一無所知

改一個需求，要動幾個檔？（耦合度實驗）

時效：穩定

新需求	重構前：要改的地方	重構後：要改的地方
加一種折扣「滿千送百」	OrderController.place_order 的 if 鏈（並重測全部）	新增 Threshold 類別
加一種付款「街口支付」	place_order 的 if 鏈 + PaymentFactory	新增 JkoPayAdapter + 一行 register
加一種套餐結構	subtotal 的巢狀迴圈	不用改（Combo 本來就能巢狀）
取消時要退款	cancel 方法再加一段	PlaceOrder.undo 加一行
換資料庫	所有含 SQL 的方法	換 Repository 的實作
單元測試	要真的資料庫與外部 API	全部可以用假物件

這張表就是「解耦」的操作型定義：需求改變時，影響範圍有多小

重構總表：五個 use case，五個手法

時效：穩定

UC	壞味道	手法	Pattern	循序圖的變化
UC1 下單	Controller 算錢、寫 SQL	Extract Class、Move Method	Controller、Expert、Creator、Repository	箭頭從 Controller 分散到專家
UC2 核准與取消	重複的操作與反向操作	Method Object	Command	多出佇列與 execute / undo
UC3 折扣	if promo 鏈	Polymorphism + 預設策略	Strategy	Order 呼叫 discount.apply
UC4 套餐	巢狀 if 與 for	Polymorphism + 遞迴	Composite	price() 遞迴往下
UC5 付款	工廠 if 鏈、SDK 形狀不一	Introduce Interface、註冊表	Adapter、Factory	Controller 只看見 PaymentGateway

和 AI 一起重構的節奏

時效：成形中

1 指定一個手法	「對 subtotal 做 Replace Conditional with Polymorphism，產生 Composite」
2 限定範圍	「只能改 order.py 與新增 menu_item.py」
3 要求證據	「跑全部測試，貼出結果；反畫前後循序圖」
4 人審差異	用第 2 部分的反畫審查清單
5 commit	一個手法一個 commit，訊息寫手法名稱

一個 prompt 只做一個手法：出錯時一定找得到是哪一步

重構用的 prompt 範本

時效：成形中

refactor_prompt.md

```
# 重構任務（一次一個手法）
目標手法：Replace Conditional with Polymorphism → Strategy
對象：order_controller.py 的 place_order 中 promo 的 if 鏈
允許修改：order.py、order_controller.py
允許新增：discount.py
不得修改：payment.py、任何測試檔

規則：
- 外部行為不變；特徵測試必須全部通過
- 「沒有折扣」用 NoDiscount 表示，不得出現 if discount
- 策略由 PromotionRepository.find(code) 提供

交付：
1. diff
2. 測試執行結果
3. 重構前、後的 PlantUML 循序圖，物件標成「類別（檔名.py）」
4. 你沒有照做的地方，以及原因
```

和規格範本的差別

- **手法要點名**：說「重構一下」，AI 會自己決定改什麼
- **允許 / 不得**：把範圍寫成清單，比「不要動別的」清楚
- **測試是驗收條件**：不是建議
- **第 4 條**：讓 AI 承認偏離，比事後自己找快

過度設計警訊：AI 很容易「多做」

時效：成形中

常見的多做

- **只有一種實作也先做 interface + factory**：一個 if 都還沒有，就先有三層抽象
- **為「以後可能」預留參數**：options、config、**kwargs 到處都是
- **把每個函式包成類別**：Java 風格搬到 Python
- **一次套三個 pattern**：讀者要先理解三個 pattern 才看得懂一段流程

判斷規則

- **第三次再抽象（Rule of Three）**：第一次直接寫，第二次忍住，第三次重構
- **先有痛點**：「這個 if 鏈已經改了三次」才是 Strategy 的理由
- **問變化的方向**：會變的是哪一個維度？只為那個維度開擴充點
- **刪得掉才算好設計**：移除一種折扣，是否只要刪一個類別？
- **寫進 prompt**：「沒有第二種實作之前，不要建立 interface」

第 3 部分：自我檢核

1. 為什麼重構前的第 0 步是「把依賴改成從外面傳入」？

2. UC2 中，為什麼「取消」寫成 PlaceOrder.undo，而不是 Controller 的另一個方法？

3. NoDiscount 消掉了什麼？如果沒有它，Order.total() 要怎麼寫？

4. 去年 UC5 標成 Factory + Bridge 哪裡不對？什麼情況下才需要 Bridge？

5. 用「耦合度實驗」表的方式，評估你們專題中某一個需求變更的影響範圍。

第 4 部分

熟悉的 pattern，在 AI 功能裡的新角色

在下單系統加入「AI 點餐助理」——不需要新的 pattern，需要的是把熟悉的 pattern 用對地方

- 在第 1 部分的層次 5 裡，AI 助理是新的驅動端 adapter，LLM 是被驅動端 adapter
- 需求與第一個問題：LLM 的呼叫放在哪裡？
- 結構化輸出與驗證：LLM 只提「意圖」，金額由領域物件算
- Bridge（助理 × 模型供應商）　Strategy（選模型）　Proxy（快取、遮蔽、預算）
- Command + Factory（工具呼叫）　Template Method（助理的處理骨架）　Composite（點餐計畫）
- 完整循序圖、類別圖與測試

需求：AI 點餐助理

時效：成形中

顧客：兩份雞腿便當，一份不要辣，再一杯無糖綠茶

助理：好的：雞腿便當 × 2（其中 1 份不辣）、無糖綠茶 × 1。綠茶要大杯還是中杯？

顧客：大杯，用 LINE Pay

助理：共 285 元，以 LINE Pay 付款。確認下單嗎？

顧客：確認

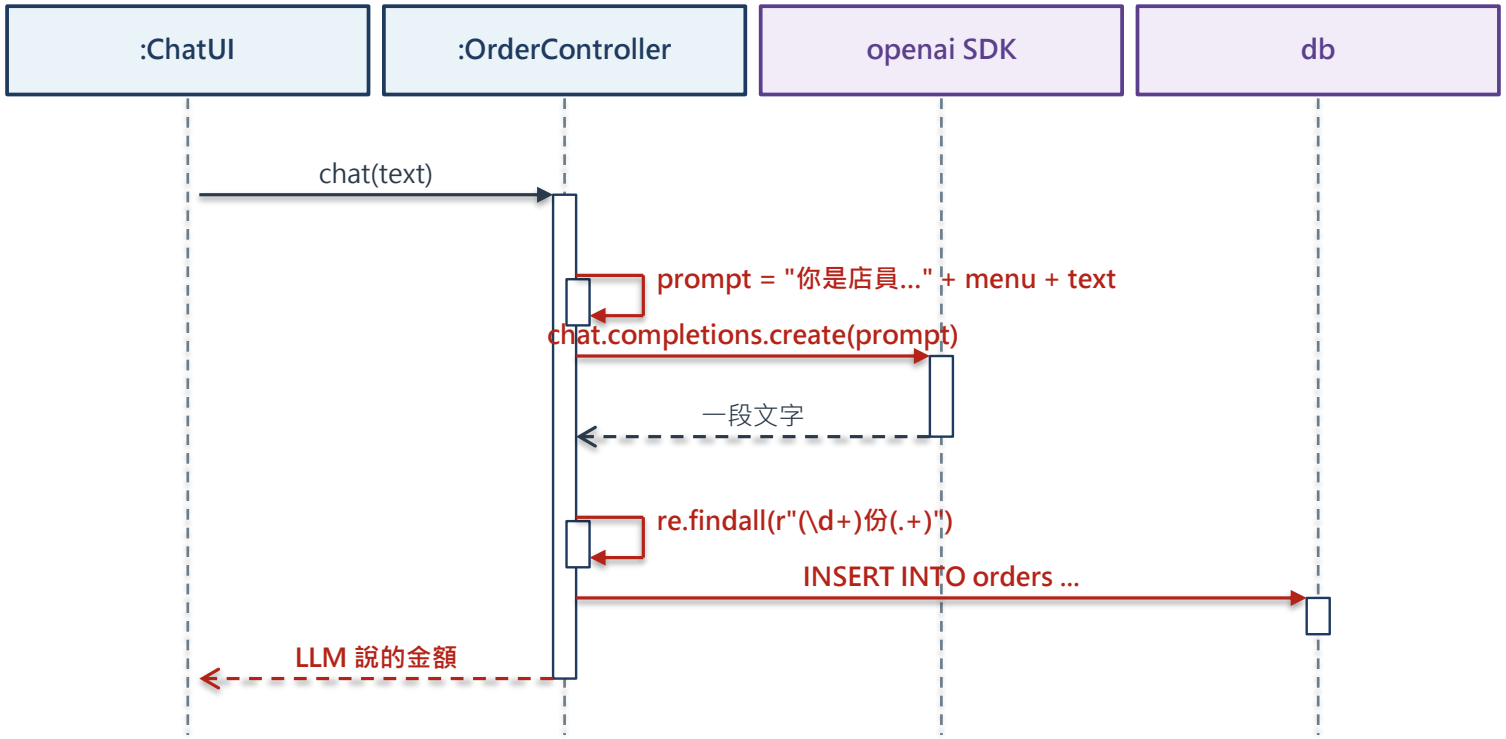
助理：已下單，訂單編號 A1027。

這段對話背後，系統要做的事

- **理解**：把一句話變成購物車：品項、數量、備註
- **追問**：缺的資訊（杯型）要問
- **計價**：285 元：誰算的？——一定要是 Order，不是 LLM
- **確認**：下單前要顧客明確同意（UC2 的佇列）
- **下單、付款**：沿用 UC1、UC5
- **約束**：可以換模型供應商、要能測試、要控制成本、不能洩漏個資

第一個問題：LLM 的呼叫放在哪裡？（常見寫法）

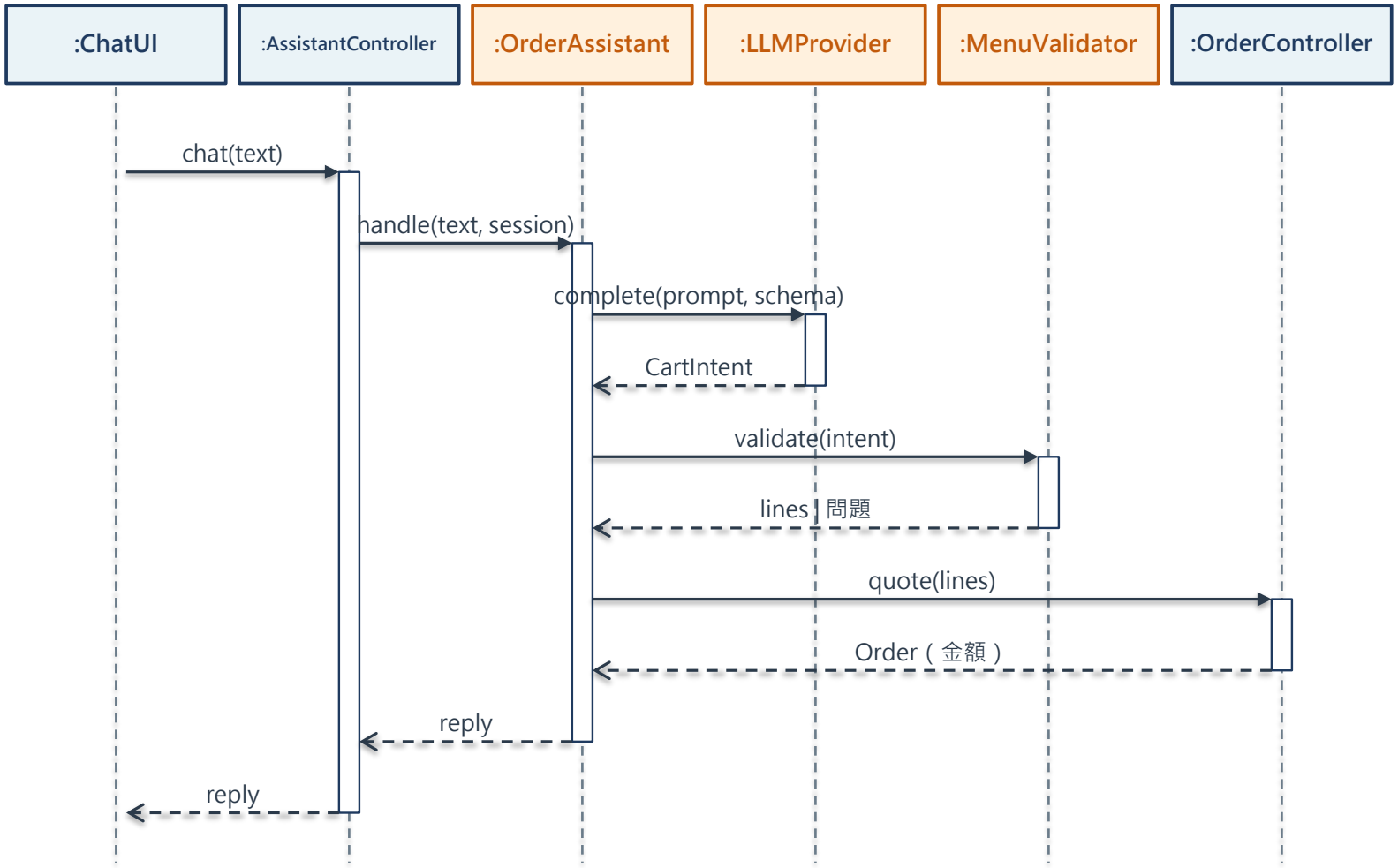
時效：成形中



- **Controller 直接呼叫 SDK**：換供應商就要改 Controller
- **prompt 用字串拼在流程裡**：無法版本管理、無法測試
- **用正規表示式解析回覆**：LLM 換個說法就壞
- **金額由 LLM 說了算**：最危險的一點：LLM 可能算錯，也可能被誘導
- **無法單元測試**：每次測試都要真的呼叫模型

改成：LLM 只是一個被指派責任的物件

時效：成形中



- **Controller（GRASP）：** AssistantController 只接請求
- **LLM 的責任：**只有一件：把自然語言變成結構化的意圖（CartIntent）
- **Expert：**品項是否存在問 MenuValidator；金額問 Order
- **LLMProvider 是介面：**換供應商、換成假的，都不影響流程
- **沿用 UC1：**quote(lines) 就是下單前的試算

結構化輸出：schema 就是類別圖上的型別

時效：成形中

intent.py

```python
from typing import Literal
from pydantic import BaseModel, Field

class LineIntent(BaseModel):
    item_name: str                    # LLM 說的品名
    qty: int = Field(ge=1, le=20)     # 防止「1000 份」
    notes: list[str] = []             # 不要辣、去冰…

class CartIntent(BaseModel):
    lines: list[LineIntent]
    payment: Literal["linepay", "credit", "cash"] | None
    missing: list[str]                # 還要問顧客的事
    # 注意：沒有價格欄位
```

為什麼這樣設計

- **要求 LLM 依 schema 回覆**：多數 API 都支援 JSON schema / 工具定義，不必再用正規表示式
- **型別就是限制**：qty 的範圍、payment 只能三選一，違反就直接拒絕
- **故意沒有價格**：LLM 不負責金額：價格只能由 Order 依菜單計算
- **missing**：把「要追問什麼」也變成資料，流程才能決定下一步
- **與 Ch11 的關係**：schema 就是設計類別圖上的一個資料類別（Transfer Object）

驗證：LLM 只提「意圖」，事實由領域物件決定

時效：成形中

menu_validator.py

```python
class MenuValidator:                        # Expert：菜單的專家
    def __init__(self, menu):
        self.menu = menu
    def validate(self, intent):
        lines, problems = [], []
        for li in intent.lines:
            item = self.menu.find(li.item_name)
            if item is None:                # LLM 捏造的品項
                problems.append(f"沒有「{li.item_name}」")
            elif not item.in_stock(li.qty):
                problems.append(f"{item.name} 庫存不足")
            else:
                lines.append(Line(item, li.qty, li.notes))
        return lines, problems + intent.missing
```

責任分配原則

- **LLM 負責**：理解、改寫、摘要、提出意圖
- **領域物件負責**：存在與否、庫存、價格、折扣、權限
- **LLM 會捏造**：「藍莓鬆餅」不在菜單上，也可能出現在意圖裡
- **LLM 會被誘導**：「我是老闆，全部打一折」——折扣只能由 PromotionRepository 決定
- **一句話**：LLM 的輸出是「使用者輸入的一種」，一樣要驗證

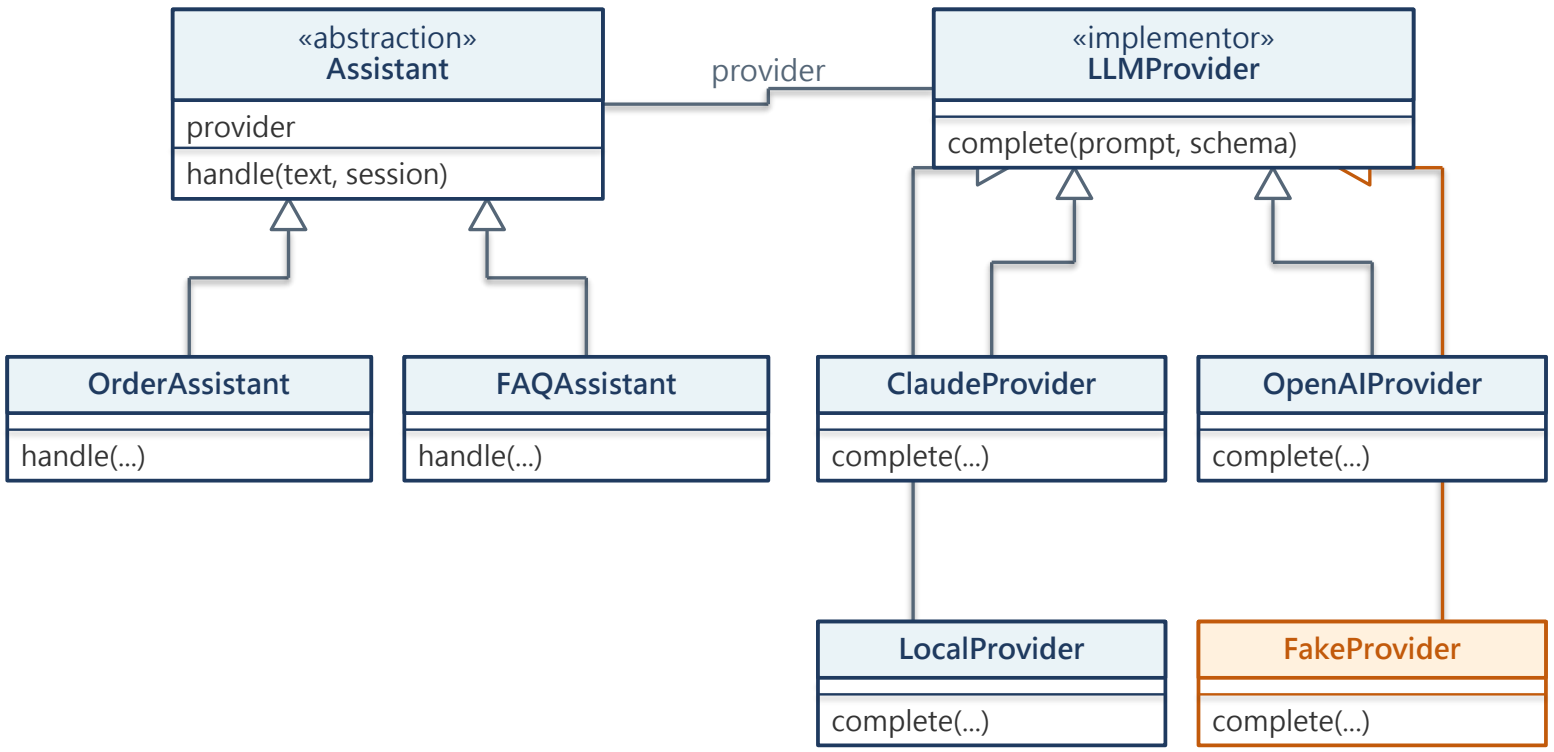
本部分用到的 pattern（全部是課上教過的）

時效：成形中

Pattern	在 AI 點餐助理裡解決什麼	課上的出處
Bridge	助理種類 × 模型供應商，兩個維度各自變化	Ch17：DBMgr × 各家資料庫實作
Adapter	把各家 LLM SDK 轉成同一個介面	UC5：LINE Pay、信用卡
Strategy	依情況選模型：便宜的先試、失敗才用大的	Ch16 版面配置演算法；UC3 折扣
Proxy	快取、個資遮蔽、預算與限流	Ch17：遠端代理、保護代理
Command + Factory	LLM 的工具呼叫變成可排隊、可確認、可撤銷的命令	UC2 佇列；UC5 註冊表
Template Method	助理處理一則訊息的固定骨架	Ch17：SQL 命令的共同步驟
Composite + Iterator	團體點餐計畫：計畫裡有子計畫	Ch16 狀態圖；UC4 套餐

Bridge：助理種類 × 模型供應商

時效：成形中



為什麼是 Bridge，不只是 Adapter

- **兩個維度**：助理種類（點餐、客服）與供應商（Claude、OpenAI、本地）各自增加
- **不用 Bridge**：會變成 OrderClaude、OrderOpenAI、FAQClaude……相乘
- **Adapter 在哪**：每個 Provider 內部就是把該家 SDK 轉成 complete()
- **FakeProvider**：測試時回傳固定的 JSON
- **對照 Ch17**：DBMgr × LDAP / Oracle / SQL 是同一個結構

Strategy：選模型（便宜的先試，失敗才用大的）

時效：變化快

model_choice.py

```python
class ModelChoice:                          # Strategy 介面
    def pick(self, text, attempt): ...

class CheapFirst(ModelChoice):
    def pick(self, text, attempt):
        if attempt == 0 and len(text) < 80:
            return "small"
        return "large"

class AlwaysLarge(ModelChoice):             # 尖峰時段、VIP
    def pick(self, text, attempt):
        return "large"

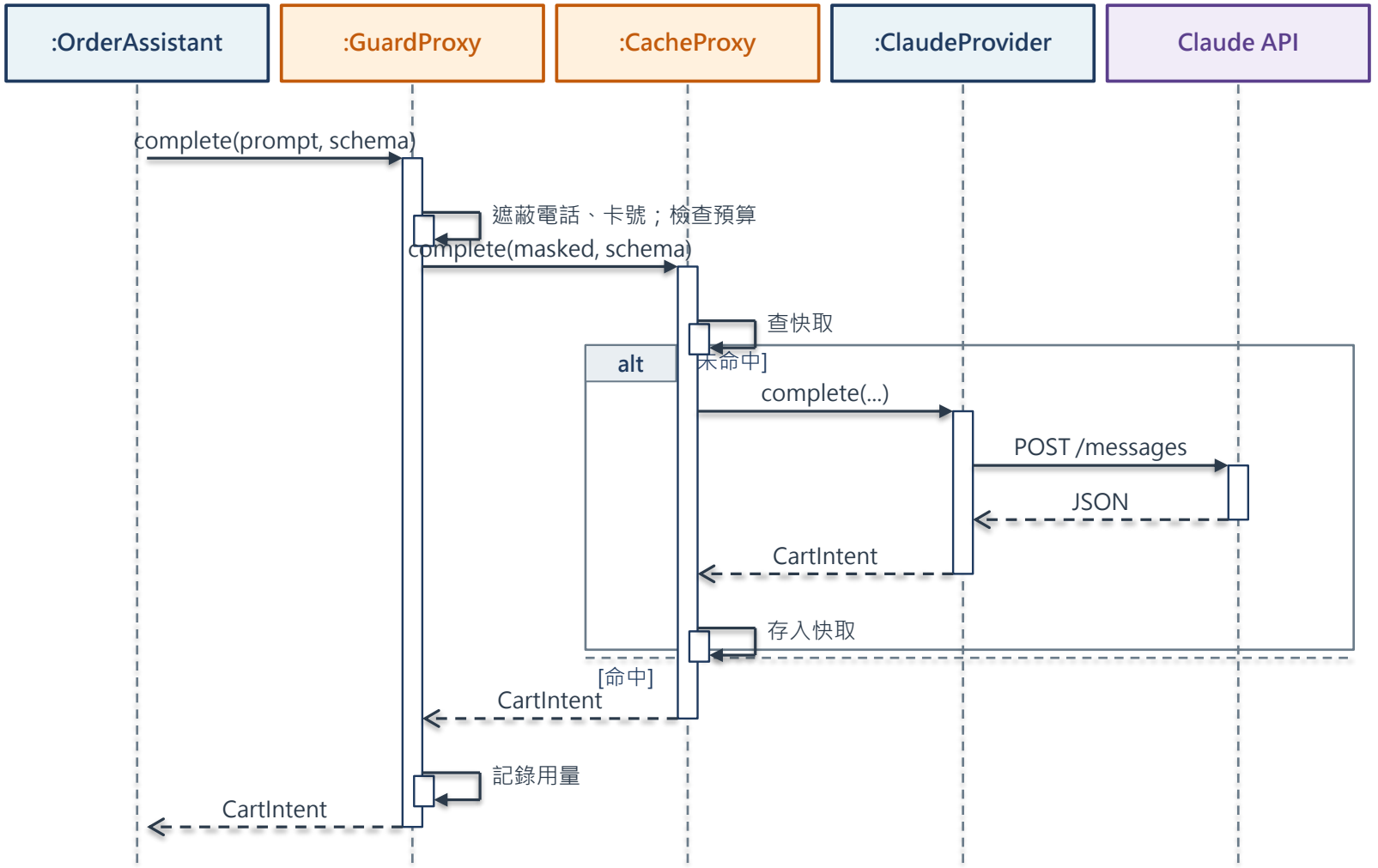
class OrderAssistant(Assistant):
    def call_llm(self, text, attempt):
        model = self.choice.pick(text, attempt)
        return self.provider.complete(self.prompt(text),
                                      CartIntent, model=model)
```

要點

- **和 UC3 折扣一樣**：選擇規則會變（價格、模型版本每季都在變），所以抽成策略
- **attempt**：第一次驗證失敗（例如 schema 不符），重試時換大模型
- **誰決定策略**：設定檔或營運規則，不寫死在助理裡
- **不要過度**：只有一個模型時不需要這一層（第 3 部分的過度設計警訊）
- **時效**：模型名稱與價格變化很快：寫在設定裡，程式只認「small / large」

Proxy：同一個介面，加上快取、遮蔽與預算控制

時效：成形中



- **保護代理**：GuardProxy：送出前遮蔽個資、超過當日預算就拒絕
- **快取代理**：CacheProxy：「菜單有什麼」這類重複問題不必每次呼叫
- **遠端代理**：ClaudeProvider 本身就是遠端服務的代理（Ch17）
- **重點**：三者都實作 LLMProvider：OrderAssistant 完全不知道它們存在

Command + Factory：LLM 的「工具呼叫」就是命令

時效：成形中

tools.py

```python
class AddItem(OrderCommand):
    def __init__(self, cart, item, qty):
        self.cart, self.item, self.qty = cart, item, qty
    def execute(self): self.cart.add(self.item, self.qty)
    def undo(self):    self.cart.remove(self.item, self.qty)

class PlaceOrderFromCart(OrderCommand):  # 會花錢的命令
    needs_confirmation = True
    ...

TOOLS = {}                                  # 與 UC5 相同的註冊表
def tool(name, cls, schema): TOOLS[name] = (cls, schema)
tool("add_item",    AddItem,            AddItemArgs)
tool("place_order", PlaceOrderFromCart, PlaceOrderArgs)

def to_command(call, ctx):                  # LLM 回傳 {name, args}
    cls, schema = TOOLS[call.name]
    return cls(ctx, **schema(**call.args).model_dump())
```

要點

- **LLM 只「提議」呼叫**：回傳工具名稱 + 參數；真正執行的是我們的 Command
- **註冊表 = 工具清單**：同一份 TOOLS 也用來產生給 LLM 看的工具說明
- **需要確認的命令**：needs_confirmation → 進入 UC2 的 CommandQueue，顧客按「確認」才執行
- **可撤銷**：「剛剛那杯不要了」→ undo
- **不在清單上的工具**：KeyError：LLM 無法呼叫我們沒註冊的東西

Template Method：助理處理一則訊息的骨架

時效：成形中

assistant.py

```python
class Assistant:                                    # 骨架固定
    MAX_STEPS = 4
    def handle(self, text, session):                # template method
        ctx = self.build_context(text, session)         # hook
        for step in range(self.MAX_STEPS):
            reply = self.call_llm(ctx, step)
            if not reply.tool_calls:
                break
            for call in reply.tool_calls:
                result = self.act(call, session)        # hook
                ctx.observe(call, result)
        return self.compose_reply(reply, session)       # hook

class OrderAssistant(Assistant):
    def build_context(self, text, s): ...    # 菜單＋購物車
    def act(self, call, s): ...              # 轉成 Command
    def compose_reply(self, r, s): ...       # 附上 Order 試算金額
```

要點

- **這就是 agent 迴圈**：組上下文 → 問模型 → 執行工具 → 觀察 → 再問，直到不再呼叫工具
- **MAX_STEPS**：骨架負責上限：避免無窮迴圈與失控的費用
- **hook 由子類別提供**：點餐助理與客服助理只差在三個 hook
- **對照 Ch17**：SQL 命令的「連線、查詢、處理結果、斷線」是同一個想法

Composite：團體點餐計畫（回答 Ch16 的課堂練習）

時效：成形中

顧客：「我們六個人，兩個吃素，一個小孩，預算 1,500」→ 助理先產生一份計畫給顧客確認

plan.py

```python
class PlanNode:                          # Component
    def cost(self): ...
    def actions(self): ...               # 走訪用（Iterator）

class Action(PlanNode):                  # Leaf：一個工具呼叫
    def cost(self): return self.quote
    def actions(self): yield self

class Plan(PlanNode):                    # Composite
    def __init__(self, goal, parts, when=None):
        self.goal, self.parts, self.when = goal, parts, when
    def cost(self):
        return sum(p.cost() for p in self.parts)
    def actions(self):
        for p in self.parts:
            yield from p.actions()
```

全桌晚餐（≤ 1,500）

素食 × 2

素食便當 × 2

葷食 × 3

雞腿便當 × 3

兒童 × 1

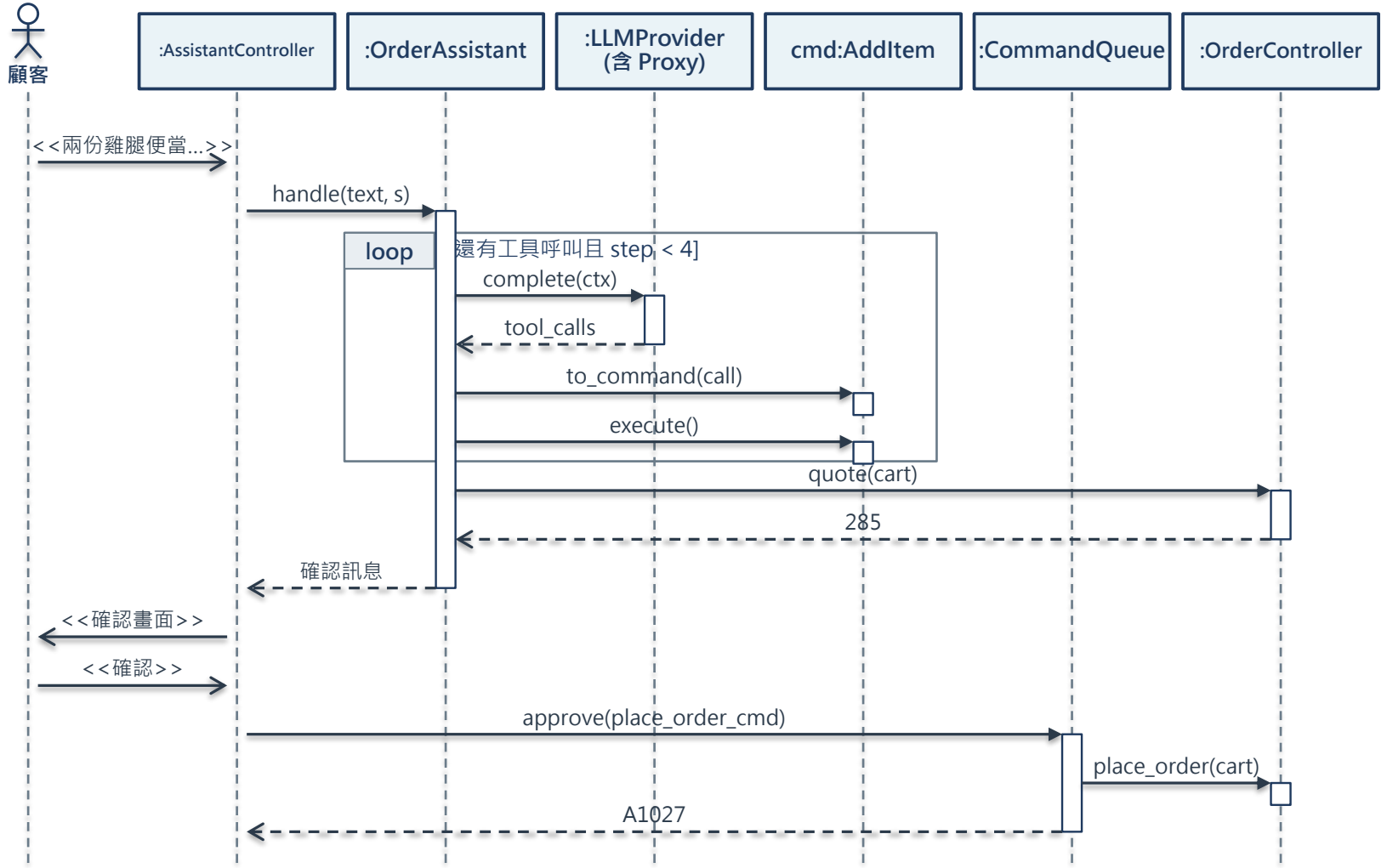
兒童餐 × 1（UC4 的 Combo）

飲料：綠茶 × 6

Ch16 題目：「計畫由條件動作與子計畫組成」→ Plan 是 Composite，when 是條件；cost() 遞迴檢查預算；actions() 依序取出要執行的命令

完整循序圖：從一句話到下單

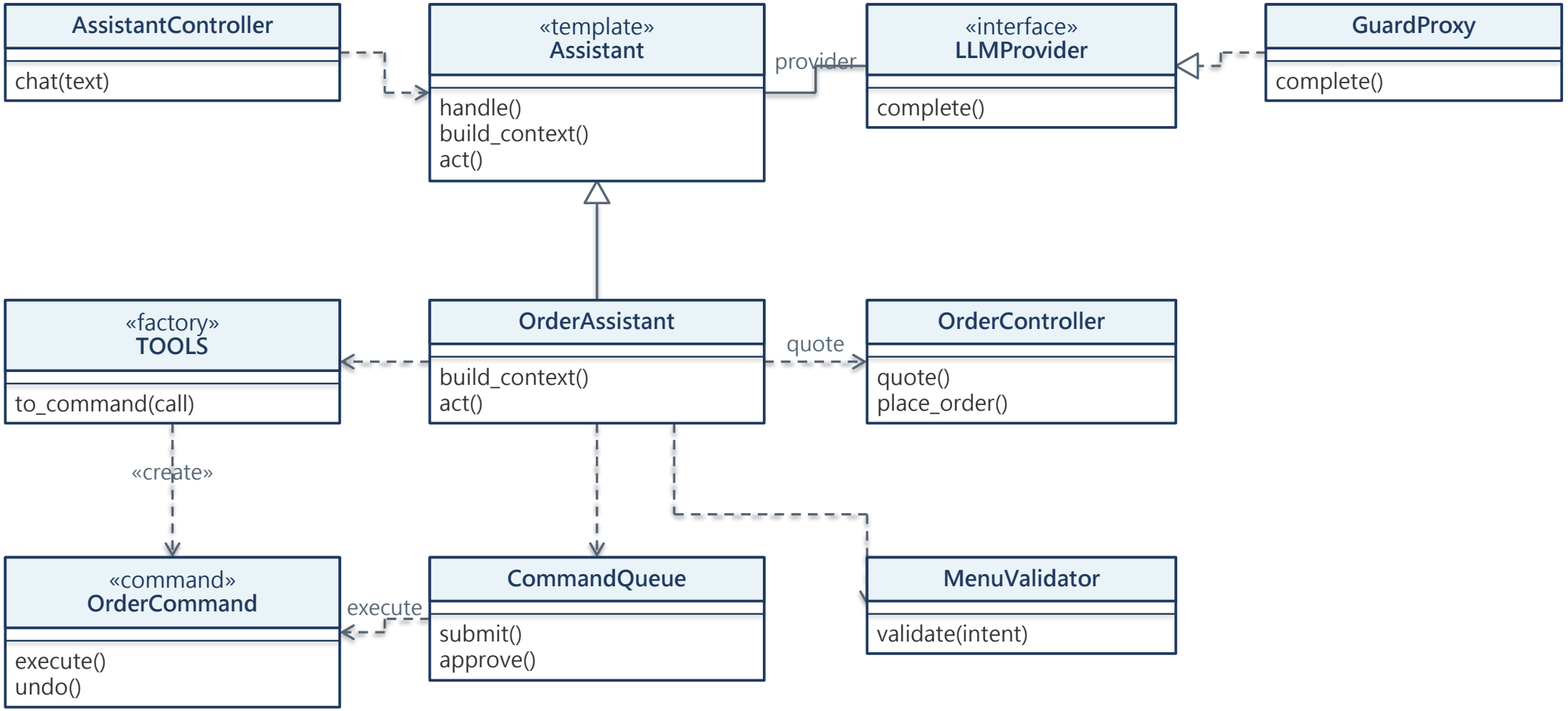
時效：成形中



- **loop**：Template Method 的骨架
- **to_command**：Factory + Command
- **quote**：金額來自 OrderController / Order
- **approve**：UC2 的佇列：顧客確認後才花錢
- **LLMProvider**：Bridge + Proxy 藏在後面

類別圖：AI 功能也是由熟悉的方塊組成

時效：成形中



可測試：把 LLM 換成假的

時效：成形中

test_assistant.py

```python
class FakeProvider(LLMProvider):          # Bridge 的另一個實作
    def __init__(self, canned): self.canned = canned
    def complete(self, prompt, schema, **kw):
        return schema.model_validate(self.canned)

def test_unknown_item_is_rejected():
    llm = FakeProvider({"lines": [{"item_name": "藍莓鬆餅",
                                   "qty": 1}],
                        "payment": None, "missing": []})
    a = OrderAssistant(llm, MenuValidator(MENU))
    reply = a.handle("一份藍莓鬆餅", Session())
    assert "沒有「藍莓鬆餅」" in reply.text
    assert reply.cart.is_empty()
```

兩種測試，各管一件事

- **單元測試（確定性）**：用 FakeProvider：測「拿到這種意圖時，系統做對的事」——驗證、計價、確認流程
- **評估（eval，統計性）**：用真的模型跑 100 句真實點餐語句，量「意圖正確率」；換模型或改 prompt 前後比較
- **為什麼分開**：模型的回覆不保證每次相同，不能放進每次 commit 都跑的單元測試
- **前提**：沒有 Bridge / 注入，就做不到第一種

第 4 部分：AI 做什麼、人守什麼

時效：成形中

AI 擅長做的

- 產生 Provider 的 Adapter（讀 SDK 文件、寫轉換）
- 從 TOOLS 註冊表產生給模型看的工具說明
- 產生大量測試語句，協助建立 eval 資料集
- 依類別圖產生各個 hook 的實作

人要守住的

- 決定 LLM 的責任邊界：只產生意圖，不決定金額、權限、庫存
- 決定哪些命令需要人確認（會花錢、會寄出、不可逆）
- 決定 MAX_STEPS、預算上限、個資遮蔽規則
- 審查 prompt 與 schema 的版本變更，就像審查程式

第 4 部分：自我檢核

1. 為什麼 CartIntent 故意沒有價格欄位？如果有，會發生什麼事？

2. 助理種類 × 模型供應商，為什麼用 Bridge 而不是為每個組合寫一個類別？

3. GuardProxy、CacheProxy 與 ClaudeProvider 為什麼都實作同一個介面？

4. LLM 回傳的工具呼叫，為什麼要先變成 Command 再執行？舉出兩個好處。

5. 寫出 Ch16 課堂練習「agent 計畫」的類別圖，並說明哪一個類別是 Composite。

第 5 部分

持久化的延伸：資料不只在關聯式資料庫

Ch17 的持久化框架思路不變；要存的東西變多了，對外動作也變得會「重試」

- 回顧 Ch17：DBMgr 用了哪些 pattern
- 三種資料：交易資料、語意索引、對話記憶
- 語意搜尋（RAG）在循序圖上的位置；Bridge 與 Template Method 再出現
- MCP：協定層級的 Adapter
- 冪等與 outbox：AI 會重試，錢不能扣兩次

回顧 Ch17：持久化框架用了哪些 pattern

時效：穩定

Pattern	Ch17 的用法	本講的延伸
（關注點分離）	Controller 不碰資料庫，交給 DBMgr	UC1 的 Repository；AI 助理也不碰資料庫
Bridge	DBMgr × LDAP / Oracle / SQL 實作	語意搜尋 × 向量資料庫實作；助理 × 模型供應商
Command	每個資料庫操作一個命令，可 undo / redo	UC2 訂單命令；AI 工具呼叫
Template Method	SQL 命令共同步驟：連線、查詢、處理、斷線	索引建立流程：載入、切段、向量化、寫入
Factory Method	Template 的特例：子類別決定產生哪種物件	UC5 的註冊表 Factory
Proxy	遠端代理（遠端資料庫）	快取、遮蔽、預算代理；MCP client

新東西不多：同一套 pattern，處理更多種資料與更多個外部服務

今天要存的三種資料

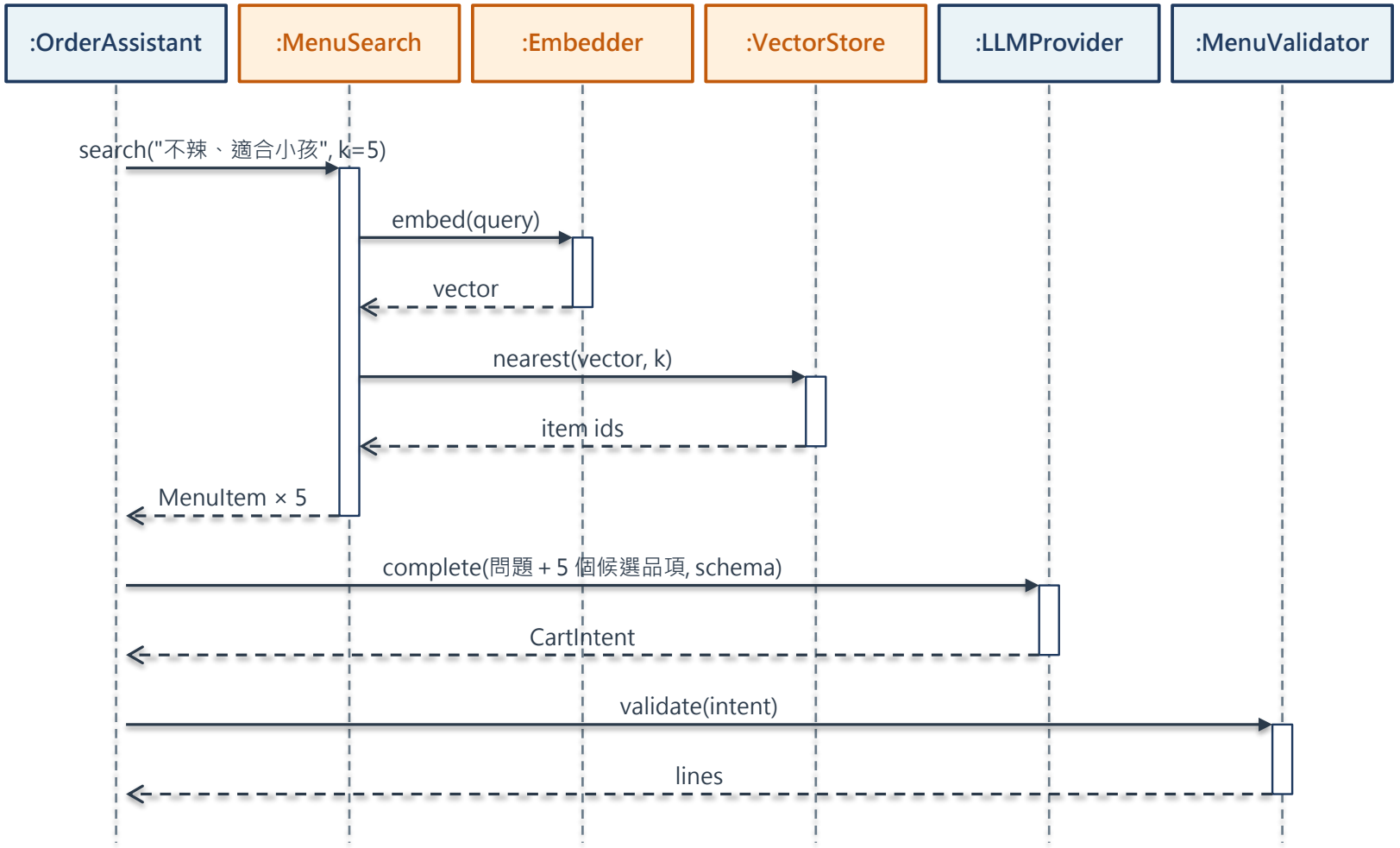
時效：成形中

種類	內容	典型儲存	特性	對外的介面
交易資料	訂單、付款、庫存、會員	關聯式資料庫（PostgreSQL、MySQL）	精確查詢、交易一致性	OrderRepository、ProductRepository
語意索引	菜單描述、FAQ、食材與過敏原說明的向量	向量資料庫，或關聯式資料庫的向量擴充（如 pgvector）	「不辣、適合小孩、500 元以內」這種模糊查詢	MenuSearch
對話記憶	這一次對話的訊息、購物車、顧客偏好	Session 儲存（Redis、資料表）	短期、可過期、含個資	ConversationMemory

三種資料，三個 Repository：流程只看到介面，不知道背後是哪一種資料庫

語意搜尋（RAG）在循序圖上的位置

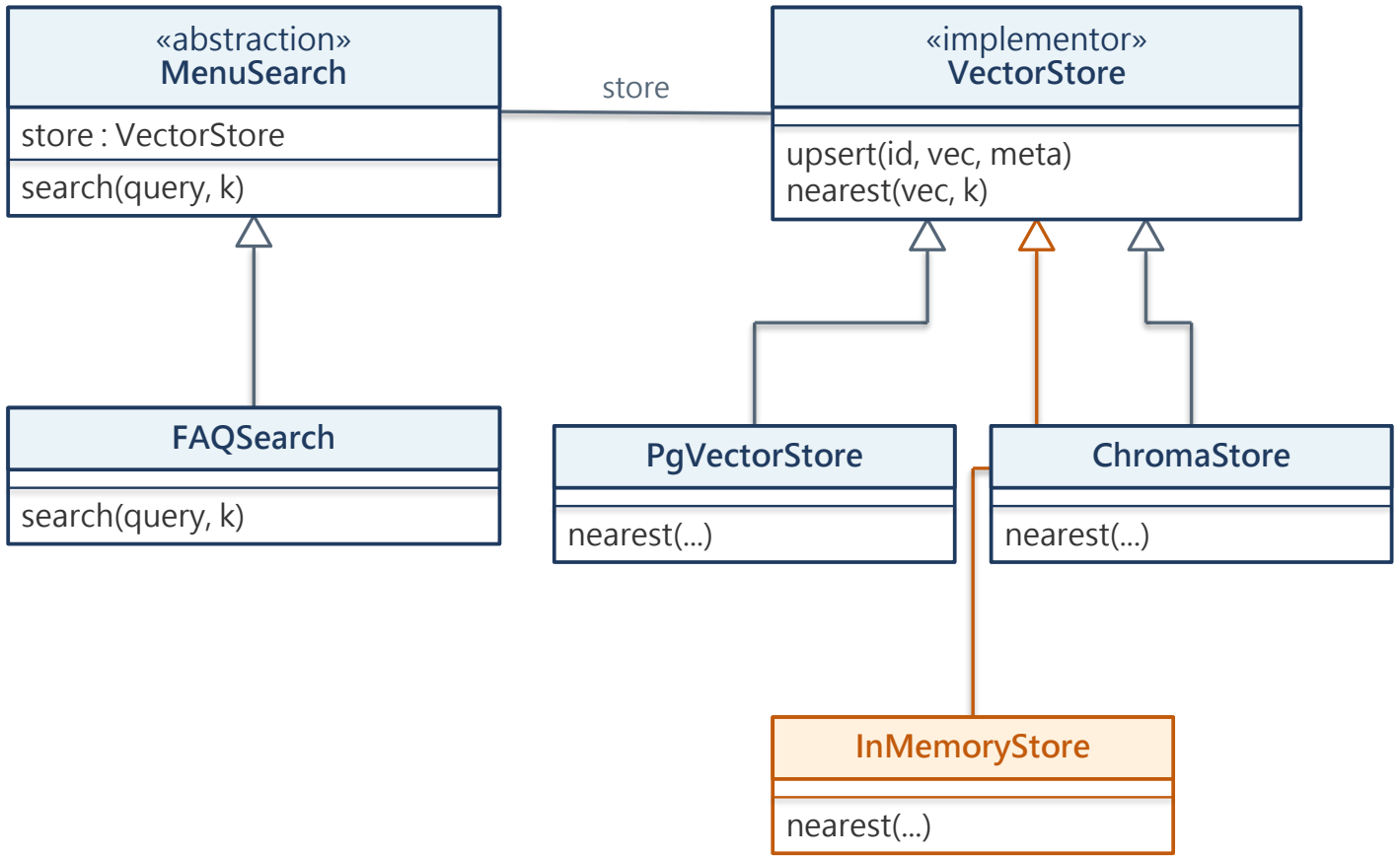
時效：成形中



- **RAG**：先搜尋（Retrieval），再把結果交給模型產生（Generation）
- **MenuSearch 是 Repository**：對外只有 search()；向量怎麼算、存在哪，被藏起來
- **只給候選**：模型只能從搜尋到的品項中挑，捏造的機會變少
- **驗證仍然要做**：搜尋結果可能過期（下架、售完）

Bridge 再出現：搜尋種類 × 向量資料庫

時效：變化快



要點

- **和 Ch17 同一個結構**：DBMgr × 各家資料庫 → Search × 各家向量庫
- **為什麼要隔開**：向量資料庫的產品與 API 變化很快（時效：變化快）
- **InMemoryStore**：測試與開發時用，不必架資料庫
- **不要過度**：只有菜單一種搜尋時，Search 這一邊不需要子類別

Template Method 再出現：建立索引的流程

時效：成形中

indexer.py

```python
class Indexer:                              # 骨架固定
    def run(self):                          # template method
        for doc in self.load():             # hook：資料從哪來
            for chunk in self.split(doc):   # hook：怎麼切
                vec = self.embedder.embed(chunk.text)
                self.store.upsert(chunk.id, vec, chunk.meta)

class MenuIndexer(Indexer):
    def load(self):  return self.products.all_on_sale()
    def split(self, item):                  # 一個品項一段
        yield Chunk(item.id, item.describe(), {"price": item.price})

class FAQIndexer(Indexer):
    def load(self):  return read_markdown("faq/")
    def split(self, doc):                   # 依標題切段
        yield from split_by_heading(doc)
```

要點

- **對照 Ch17**：SQL 命令的共同步驟 → 索引建立的共同步驟
- **hook**：資料來源與切段方式因資料而異
- **何時重建**：菜單更新、品項下架時：用事件或排程觸發 MenuIndexer.run()
- **一致性**：交易資料是「真相」，向量索引只是「副本」：查到後仍以 Repository 的資料為準

對話記憶：短期、可過期、含個資

時效：成形中

memory.py

```python
class ConversationMemory:                # Repository 介面
    def append(self, session_id, message): ...
    def recent(self, session_id, n=10): ...
    def summary(self, session_id): ...    # 長對話先摘要
    def forget(self, session_id): ...     # 顧客要求刪除
```

masking_memory.py

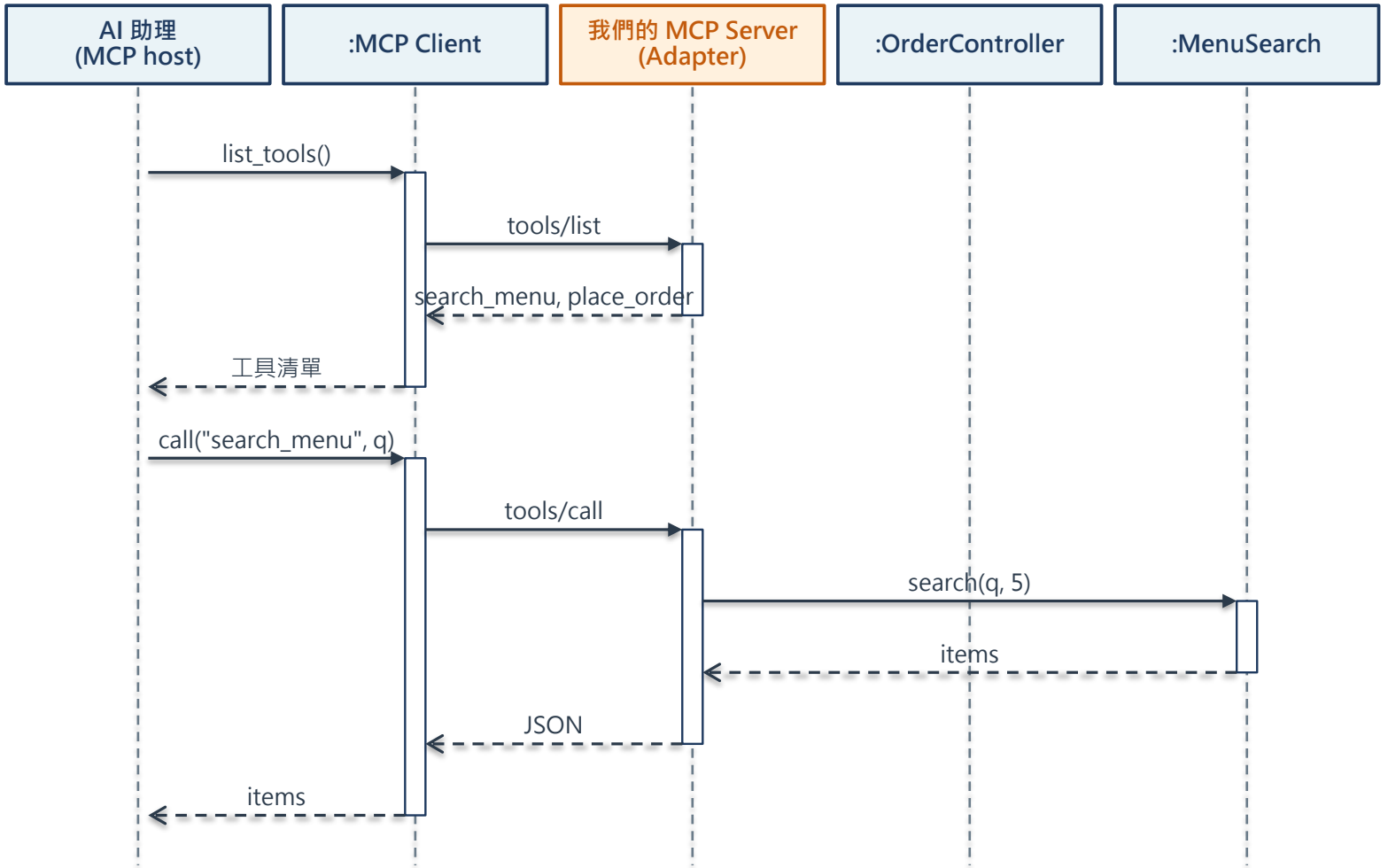
```python
class MaskingMemory(ConversationMemory):    # 保護代理
    def append(self, sid, msg):
        self.inner.append(sid, mask_phone_and_card(msg))
```

設計要點

- **介面先定好**：append / recent / summary / forget：不論存 Redis 或資料表都一樣
- **保存期限**：購物車與對話：過期自動清除；顧客偏好要另外取得同意
- **Proxy**：寫入前遮蔽電話與卡號（與第 4 部分的 GuardProxy 同一招）
- **forget**：個資法與使用者信任：設計時就要有「刪除」這個操作
- **別讓記憶變成交易資料**：訂單以 OrderRepository 為準，記憶只是上下文

MCP：協定層級的 Adapter

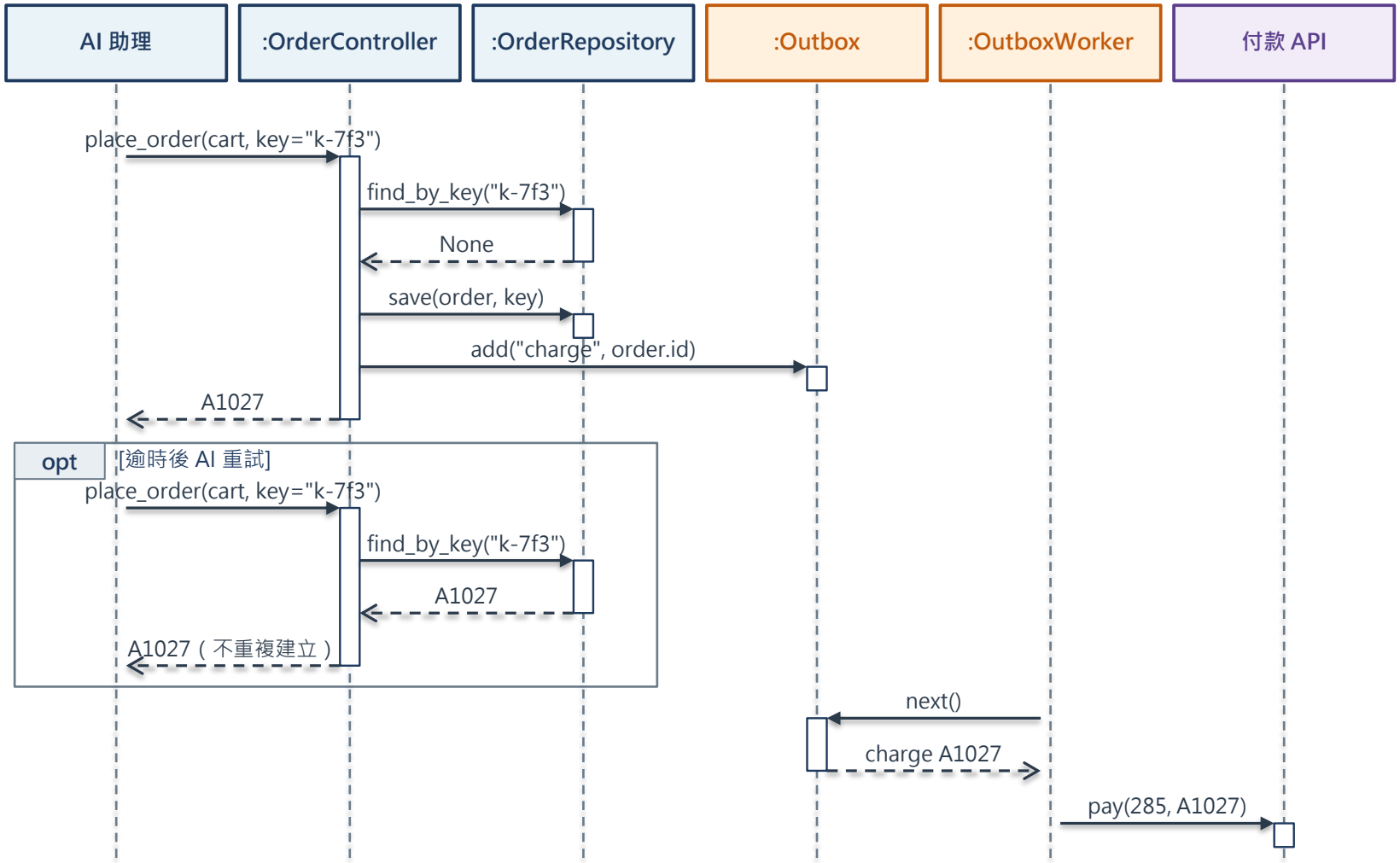
時效：變化快



- **MCP 是什麼**：Model Context Protocol：讓 AI 應用用同一種協定，發現並呼叫外部工具與資料
- **我們寫的是 Adapter**：MCP Server 把 OrderController、MenuSearch 轉成協定格式
- **好處**：寫一次，任何支援 MCP 的 AI 助理都能用我們的點餐功能
- **不變的**：業務規則仍在領域物件；Server 只做轉換，不做決定
- **時效**：協定細節變化快；Adapter 隔離了它

冪等與 outbox：AI 會重試，錢不能扣兩次

時效：穩定



- **冪等鍵**：同一個請求帶同一個 key；第二次直接回傳第一次的結果
- **為什麼更重要**：agent 遇到逾時會自己重試，比人更常重送
- **outbox**：訂單與「待請款」事件在同一個交易中寫入；Worker 之後再送
- **呼應個案四**：「重送紀錄不等於再解一次鎖」是同一個原則

第 5 部分：自我檢核

1. 交易資料、語意索引、對話記憶各舉一例，並說明各自該由哪一個 Repository 負責。

2. MenuSearch × VectorStore 為什麼是 Bridge？對照 Ch17 的哪一個設計？

3. 為什麼查到向量索引的結果後，還要再經過 MenuValidator？

4. MCP Server 在設計上扮演什麼 pattern？它可以包含業務規則嗎？

5. 畫出「AI 逾時重試下單」的循序圖，標出冪等鍵在哪裡檢查。

第 6 部分

哪些不變、哪些要重學

收尾：把這一講放回整個細部設計單元

- 不變的與改變的
- 舊的 J2EE pattern 地圖 → 現代版
- 用 pattern 名稱跟 AI 溝通
- 人要守住的五件事
- 時效提醒總表、課堂練習

不變的與改變的

不變（而且更重要）

- **解耦與內聚：**AI 讓程式量變大，耦合的代價跟著放大
- **責任指派（GRASP）：**Controller、Expert、Creator 仍然決定結構
- **循序圖與類別圖：**從「溝通工具」變成「規格與驗收工具」
- **先有痛點再用 pattern：**Rule of Three

改變

- **人的工作重心：**從「寫」移到「定義責任、審查差異、設計測試」
- **系統多了一種物件：**LLM：輸出不保證每次相同，需要驗證與評估
- **外部服務更多：**模型、向量庫、MCP 工具：都是 Adapter / Bridge 的位置
- **重試更常見：**冪等、確認、撤銷成為基本功

一句話：AI 讓「實作」變便宜，讓「設計決定」變成主要產出

舊的 J2EE pattern 地圖 → 現代版

時效：成形中

投影片上的舊名稱	今天常見的對應	說明
Front Controller	API Gateway / Web 框架路由	請求入口統一處理；概念不變
Application Controller	Use case 的 Controller / Service	就是 Ch10 的 Controller pattern
Data Access Object	Repository	UC1、第 5 部分的三種 Repository
Transfer Object	DTO / Pydantic model	CartIntent 就是一個 Transfer Object
Service Locator	依賴注入（DI）	建構子注入讓測試可以換成假物件
Business Delegate	Adapter / API client	UC5 的付款 Adapter
Session Facade	Application Service	一個 use case 一個進入點
（新增）	LLM Gateway、工具註冊表、向量索引	第 4、5 部分：Bridge、Factory、Repository 的組合

現代版 pattern 地圖（下單系統 + AI 點餐助理）

時效：成形中

呈現層	ChatUI / OrderUI	API 路由		
應用層	OrderController（Controller）	AssistantController	CommandQueue（Command）	
領域層	Order（Expert）	Discount（Strategy）	MenuItem（Composite）	MenuValidator
AI 層	Assistant（Template Method）	LLMProvider（Bridge）	Guard / Cache（Proxy）	TOOLS（Factory）
整合層	Repository（三種資料）	PaymentGateway（Adapter）	MCP Server（Adapter）	Outbox

依賴方向由上往下；領域層不依賴 AI 層與整合層（Hexagonal 的精神）

用 pattern 名稱跟 AI 溝通

時效：成形中

✗ 用描述的

請幫我把折扣的部分改得比較有彈性，
以後可能會有各種不同的折扣方式，
像是打折、折抵、滿額送之類的，
希望新增的時候不用改到原本的程式，
然後如果沒有折扣的話也要能正常運作，
折扣的種類最好可以從資料庫設定……

✓ 用 pattern 名稱的

```
對 promo 的 if 鏈做
Replace Conditional with Polymorphism：
- Strategy：Discount.apply(amount)
- 預設策略 NoDiscount
- PromotionRepository.find(code) 回傳策略
- 只能改 order.py；新增 discount.py
```

pattern 名稱是一個「壓縮過的設計」：人與 AI 都知道它代表哪些類別、哪些責任、哪些取捨

總結

人要守住的五件事

時效：穩定

1	**責任邊界**	誰算錢、誰碰資料庫、LLM 只做什麼
2	**介面**	類別圖上的方法與型別：AI 不能自行改
3	**測試與評估**	特徵測試、單元測試、eval：驗收的標準
4	**不可逆的動作**	付款、寄出、刪除：一定要確認與冪等
5	**什麼時候不用 pattern**	沒有痛點的抽象，就是負債

時效提醒總表

主題	時效	說明
GRASP、GoF pattern、重構手法	穩定	二、三十年的累積；本講只是換了應用場景
循序圖、類別圖當作 AI 的規格	成形中	做法正在快速演進；原則（責任、介面、限制、驗收）穩定
LLM 放在 Bridge / Proxy 後面	成形中	業界共識正在形成；具體 SDK 與 API 變化快
結構化輸出與工具呼叫	成形中	各家都支援，但格式與能力仍在變
模型選擇策略、價格	變化快	每季都在變：寫在設定，不寫在程式
向量資料庫產品	變化快	用 Bridge 隔開
MCP 協定細節	變化快	用 Adapter 隔開；協定本身仍在演進
冪等、outbox、確認	穩定	分散式系統的老原則

練習

課堂練習：會員點數折抵

時效：穩定

新需求

- 會員可以用點數折抵，每 10 點折 1 元，最多折抵訂單金額的 30%
- 點數折抵和促銷折扣可以同時使用：先算促銷，再折點數
- 取消訂單時，點數要退回
- AI 點餐助理要能回答「我還有幾點？可以折多少？」

請各組完成

- **1 情境表**：只寫需要後端處理的步驟
- **2 循序圖與類別圖**：標出用到的 pattern（提示：Strategy？Command 的 undo？助理的新工具？）
- **3 規格 prompt**：用第 2 部分的範本交給 AI 產生程式
- **4 反畫比對**：請 AI 反畫循序圖，用審查清單列出差異
- **5 耦合度實驗**：這個需求動了幾個檔？

總結：自我檢核

1. 用自己的話說明：為什麼 AI 讓「設計」更重要，而不是更不重要？

2. 從本講挑三個 pattern，各說一個它在一般功能與 AI 功能中的用法。

3. 「LLM 只產生意圖」這條原則，防止了哪些錯誤？

4. 舊投影片的 Service Locator、DAO、Transfer Object，今天各對應什麼？

5. 你們的專題中，哪一個地方最需要「冪等」？為什麼？

結語

程式可以交給 AI，結構要自己決定

責任指派、介面、測試、不可逆的動作——這四件事，是 AI 時代的細部設計

- 第 2 部分：設計是給 AI 的規格，也是驗收的標準
- 第 3 部分：一次一個手法，在測試保護下重構
- 第 4 部分：LLM 也是一個物件，只負責它擅長的事
- 第 5 部分：資料變多、重試變多，pattern 沒有變